Introduzione all'uso di MySQL/MariaDB e di DBeaver

Basi di dati

Riccardo Billero
riccardo.billero@unitn.it
A.A. 2026/2027

UNIVERSITÀ DI TRENTO

MySQL

- MySQL è un relational database management system (RDBMS) composto da un client a riga di comando e un server
- È un software libero pubblicato a doppia licenza, compresa la GNU General Public License, sviluppato per essere il più possibile conforme agli standard ANSI SQL e ODBC SQL
- Supportato da molti linguaggi di programmazione (inclusi Java, PHP, Python)

MySQL e MariaDB

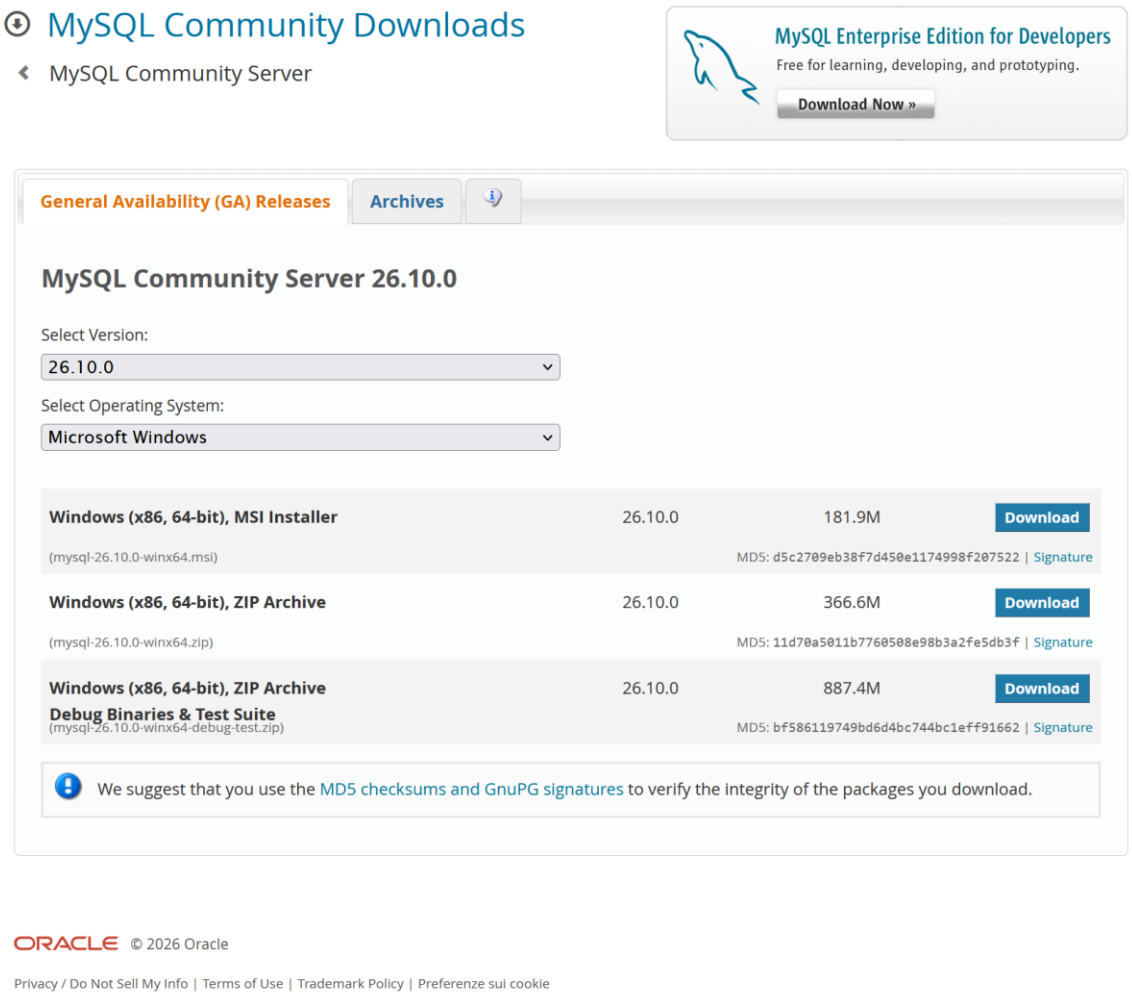
- Nasce nel 1995, sviluppato dall'azienda svedese MySQL AB
- Nel 2008 Sun Microsystems acquisisce la società svedese
- Nel 2010 Oracle acquista a sua volta Sun
  - Il prodotto viene rinominato Oracle MySQL
- Un gruppo di programmatori realizza un fork di MySQL, che viene chiamato MariaDB
  - Ad oggi molte distribuzioni Linux includono per default MariaDB, anziché MySQL
- Una pagina apposita indica le differenze tra i due progetti
  - https://mariadb.com/docs/release-notes/community-server/about/compatibility-and-differences

INSTALLAZIONE DI MYSQL

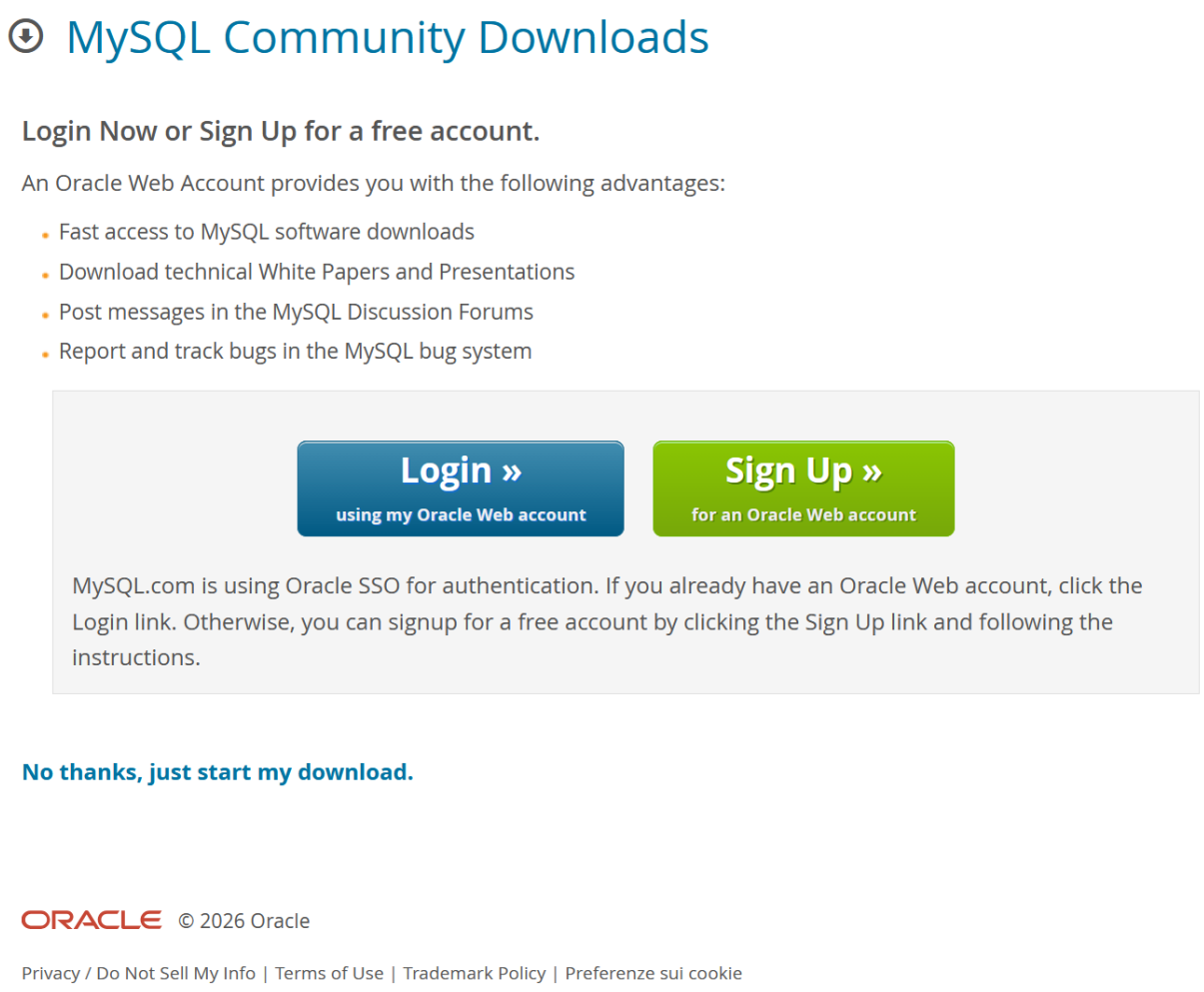
MySQL

- MySQL Community Server
  - Versione liberamente scaricabile del motore di database MySQL
  - Disponibile per Windows, macOS, Linux
  - Anche per container Docker
  - Licenza GPL
- Su Windows, scarichiamo il pacchetto MSI
  - Include MySQL Configurator per la configurazione iniziale
  - https://dev.mysql.com/downloads/mysql/

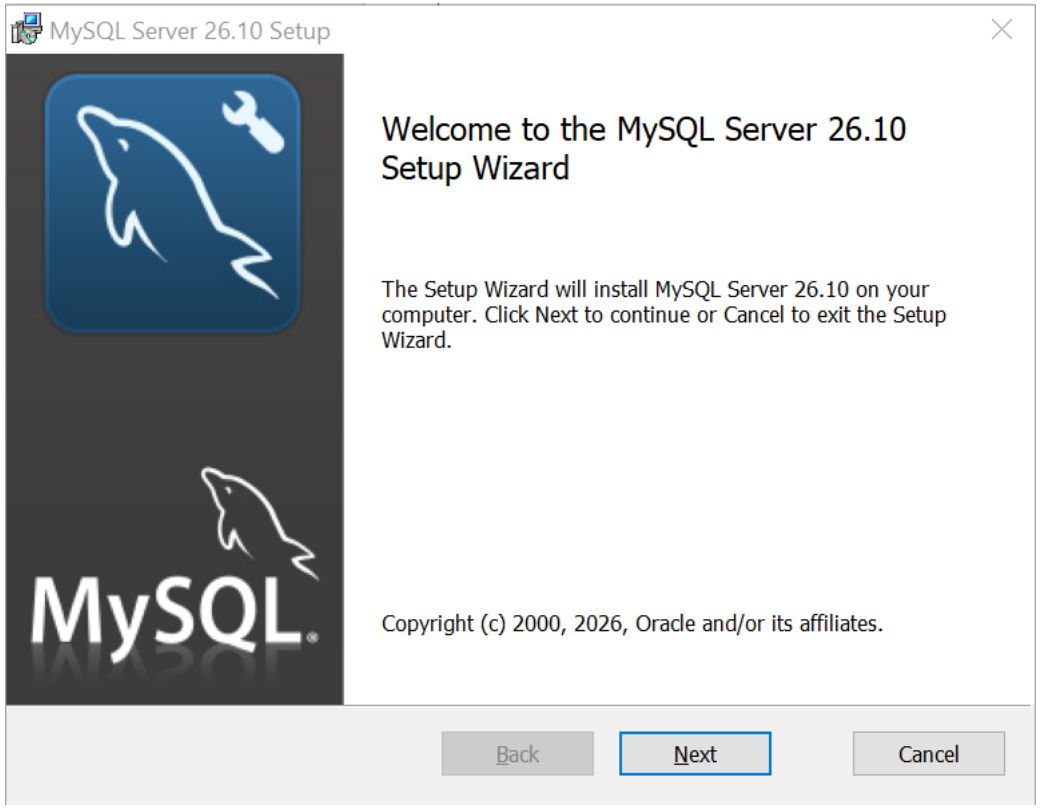
Download di MySQL (1)



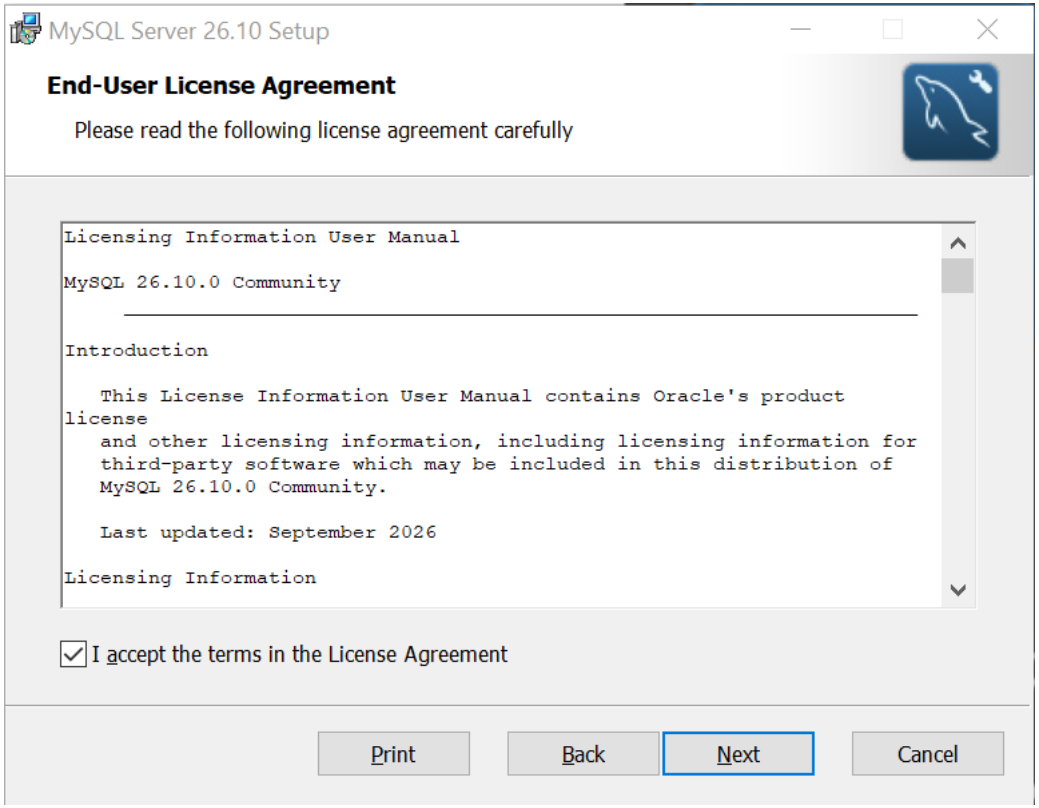
Download di MySQL (2)



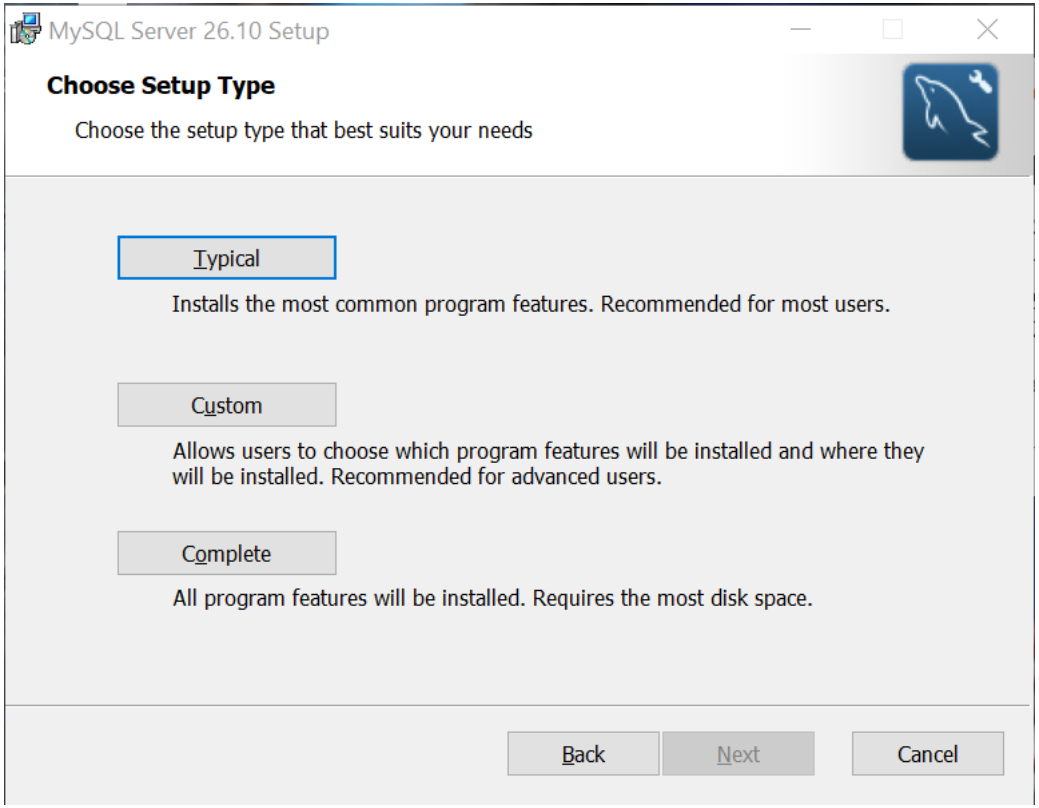
Installazione di MySQL (1)



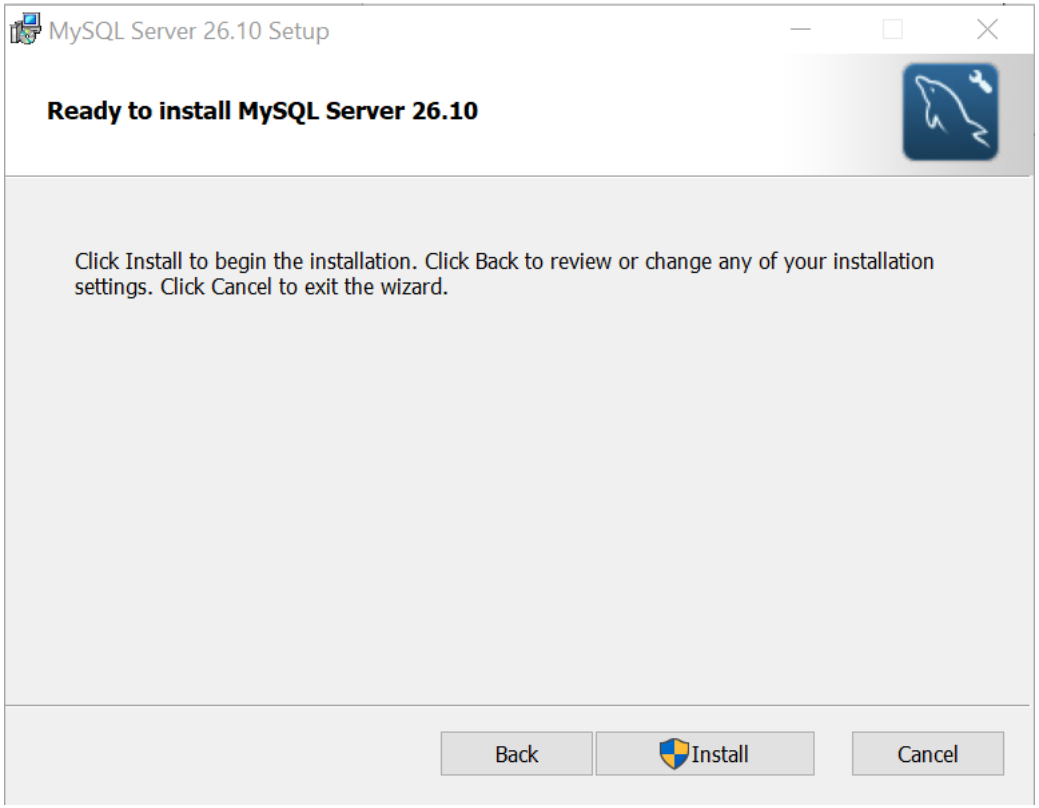
Installazione di MySQL (2)



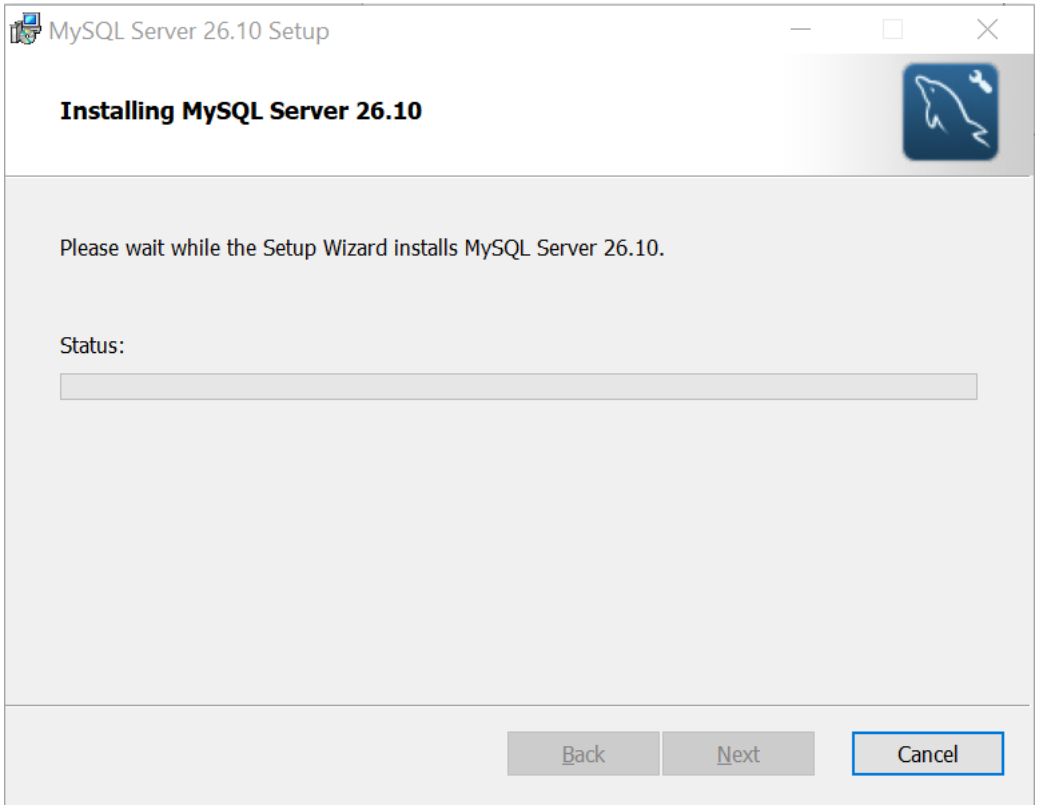
Installazione di MySQL (3)



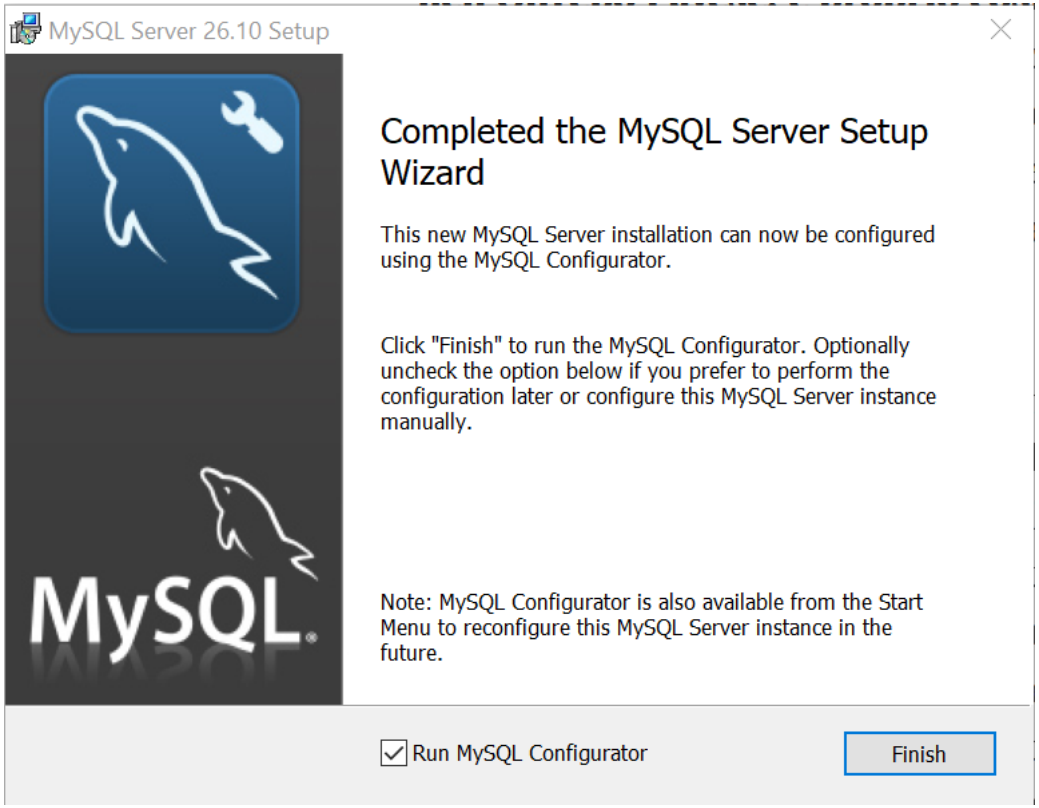
Installazione di MySQL (4)



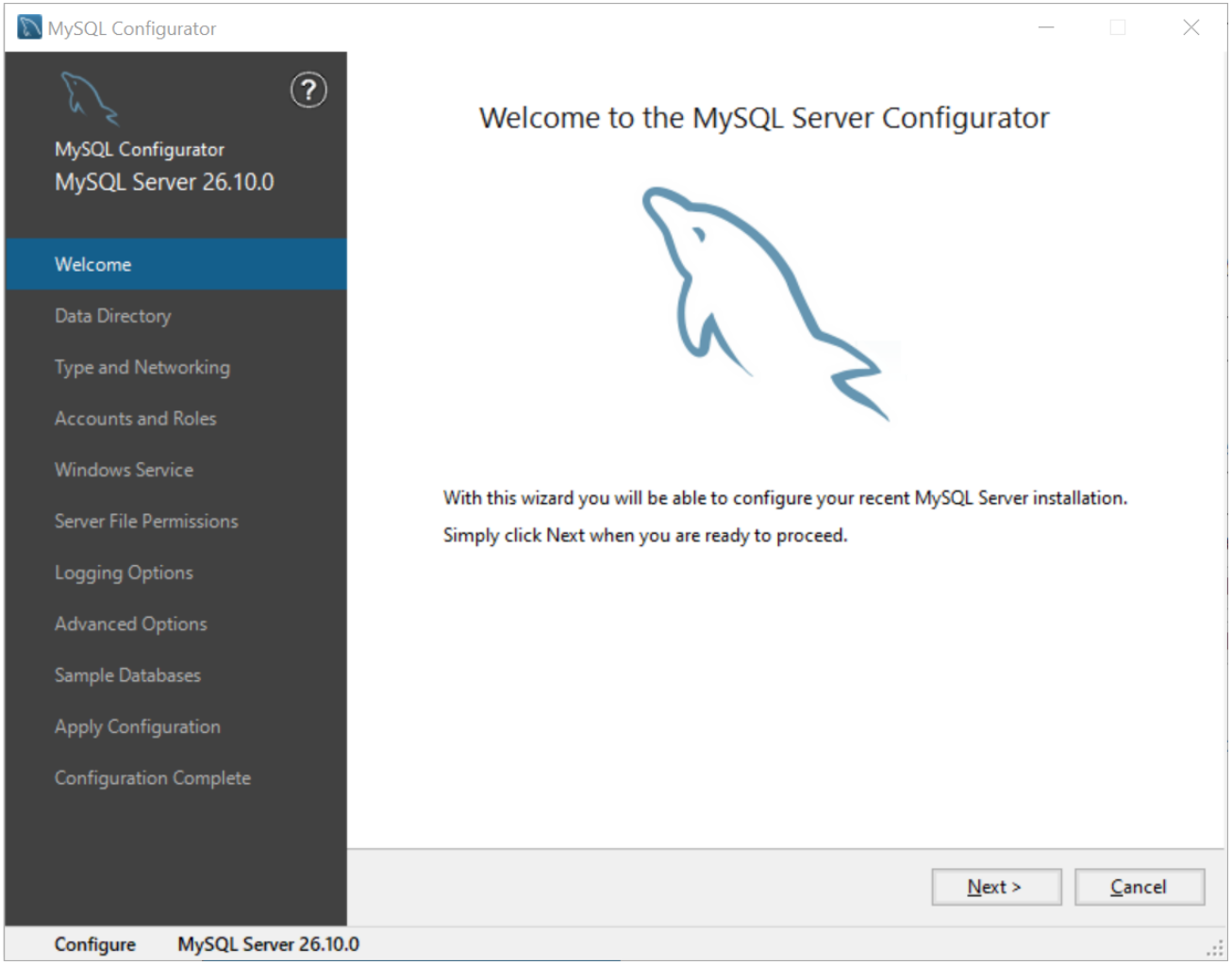
Installazione di MySQL (5)



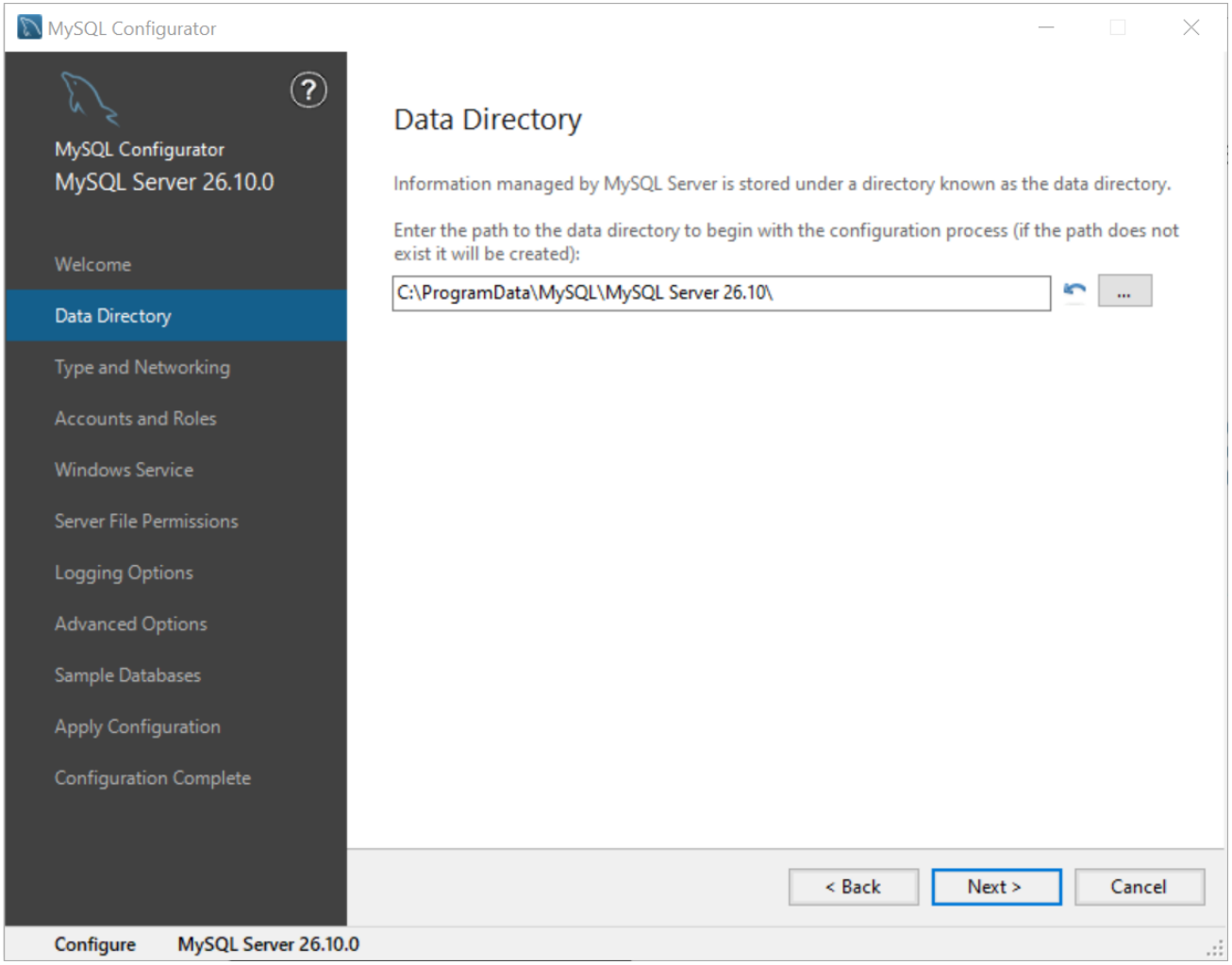
Installazione di MySQL (6)



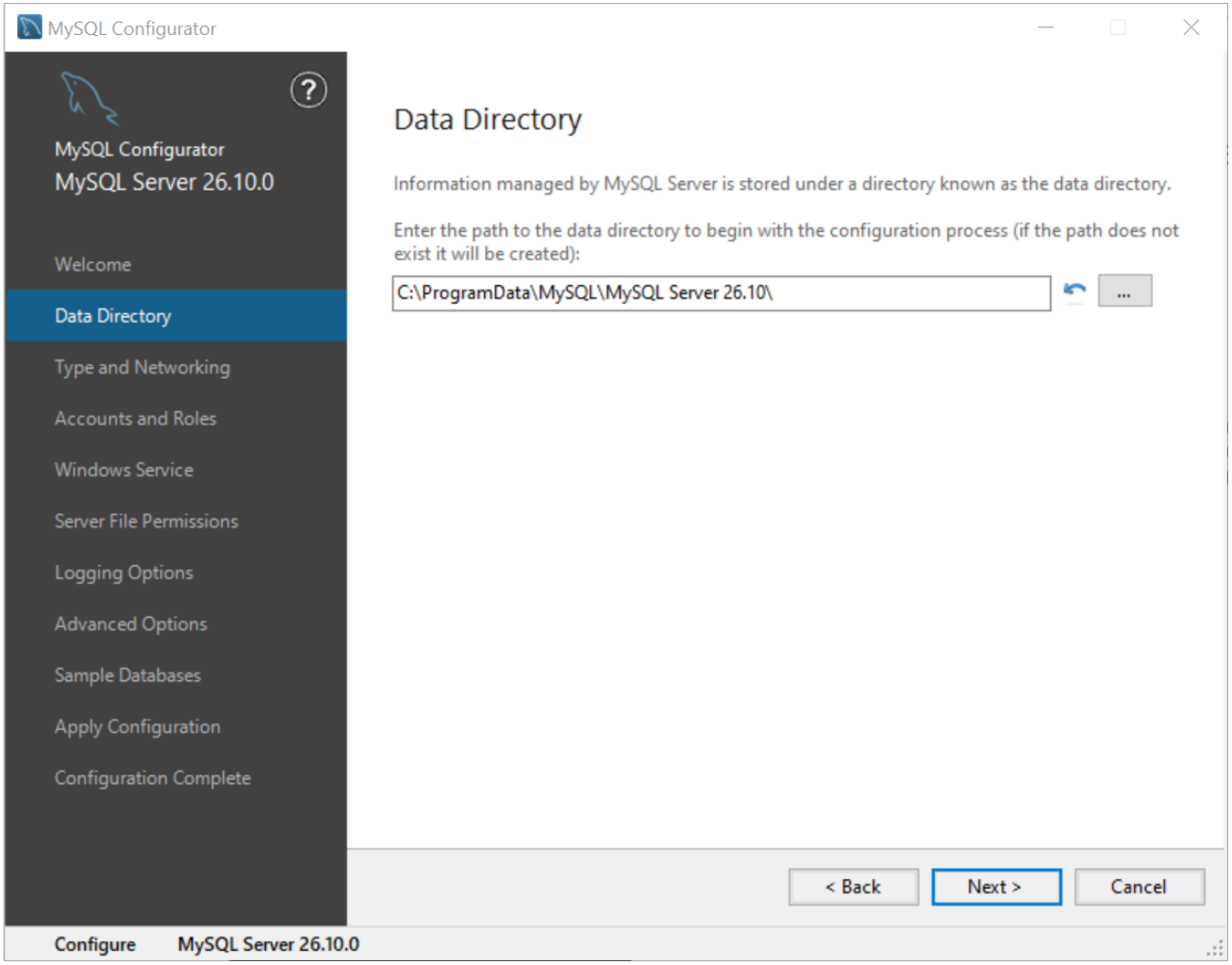
MySQL Configurator (1)



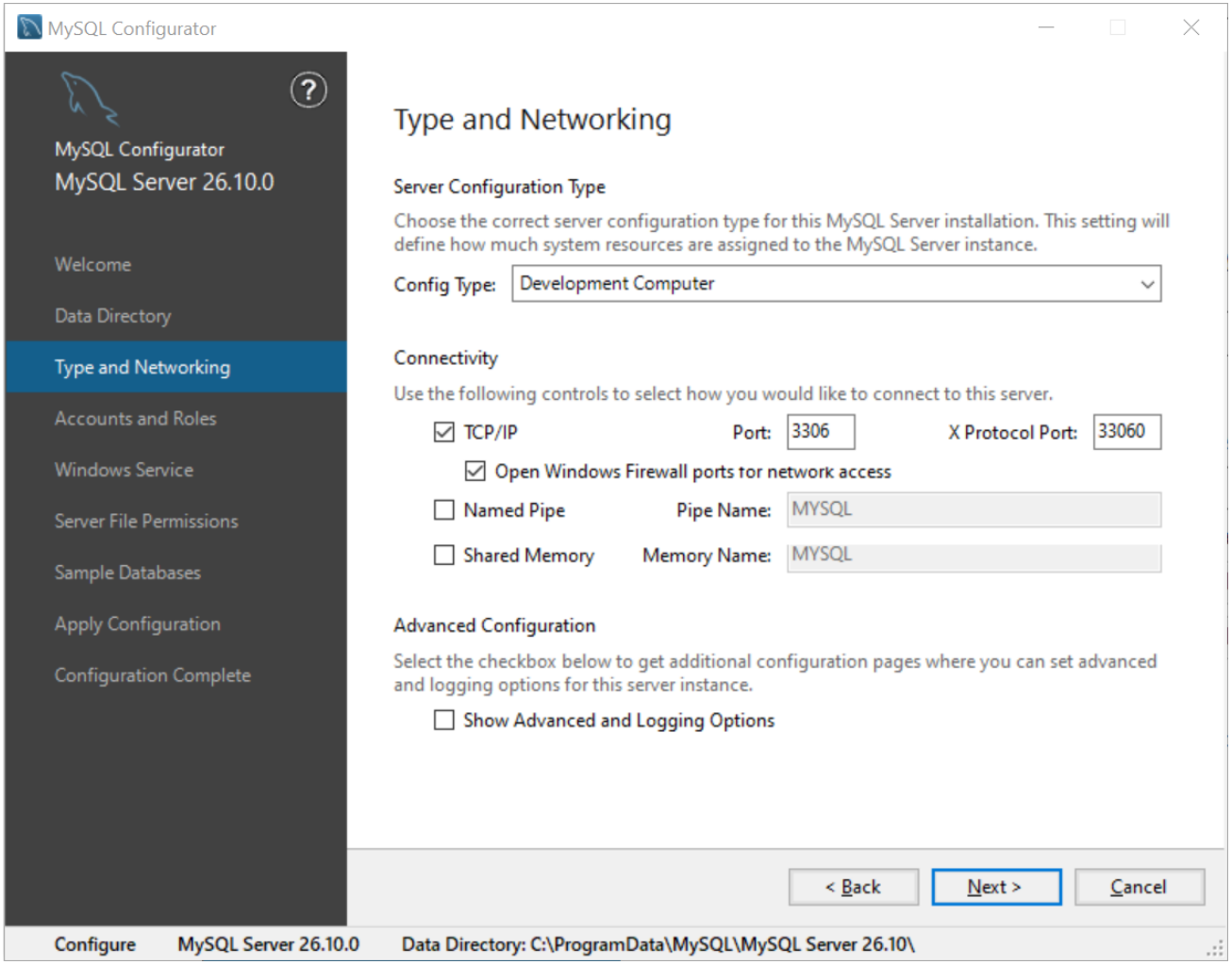
MySQL Configurator (2)



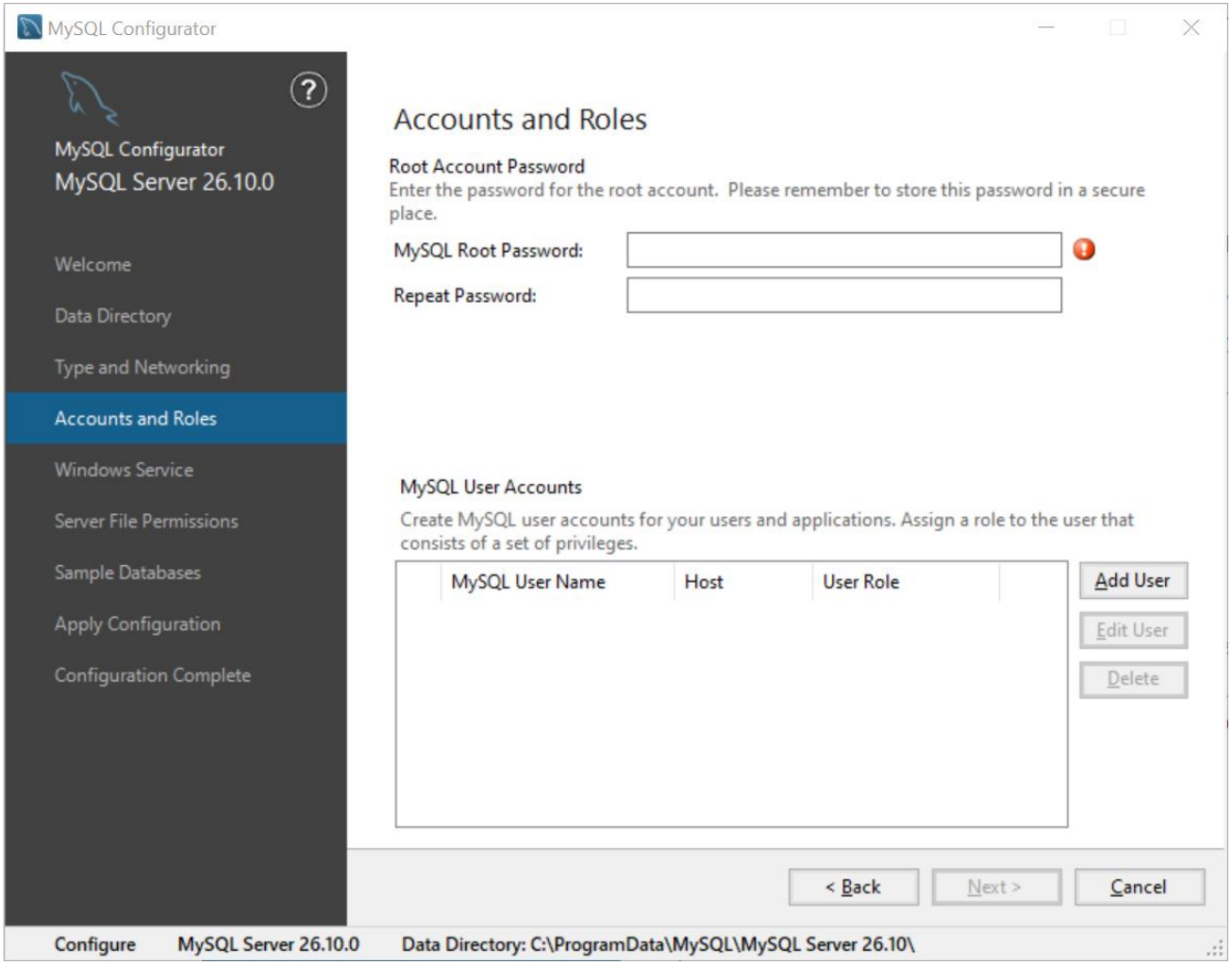
MySQL Configurator (2)



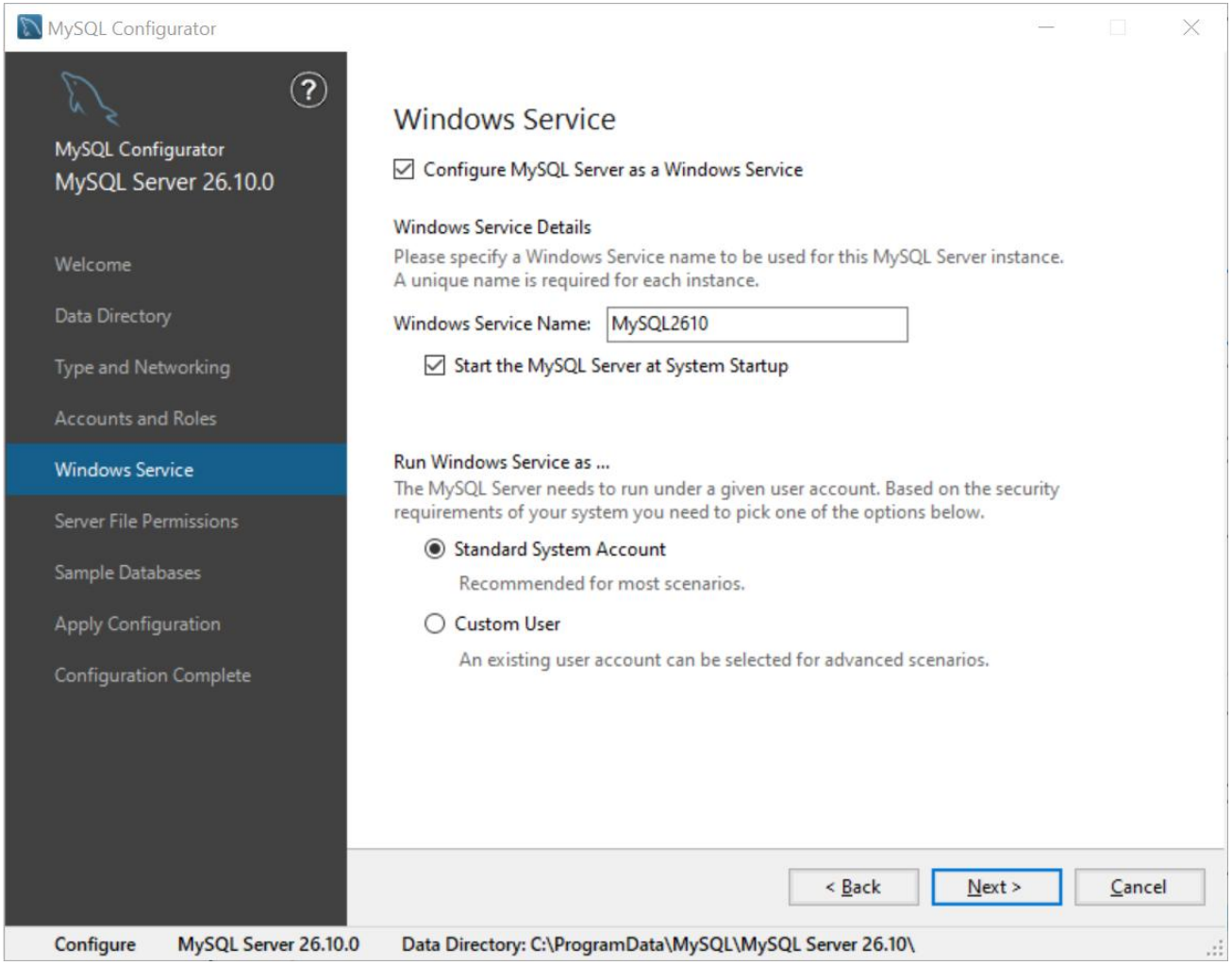
MySQL Configurator (3)



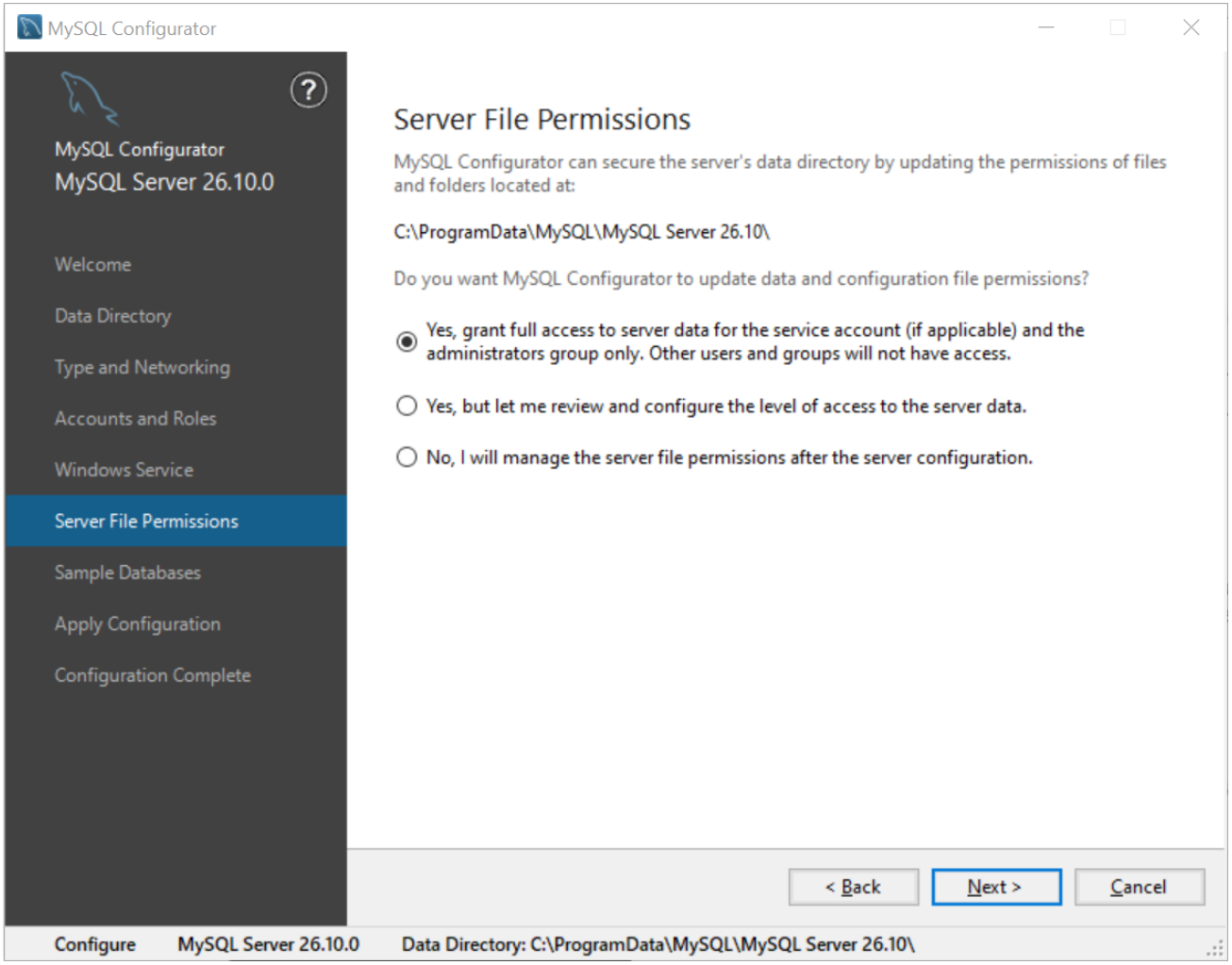
MySQL Configurator (4)



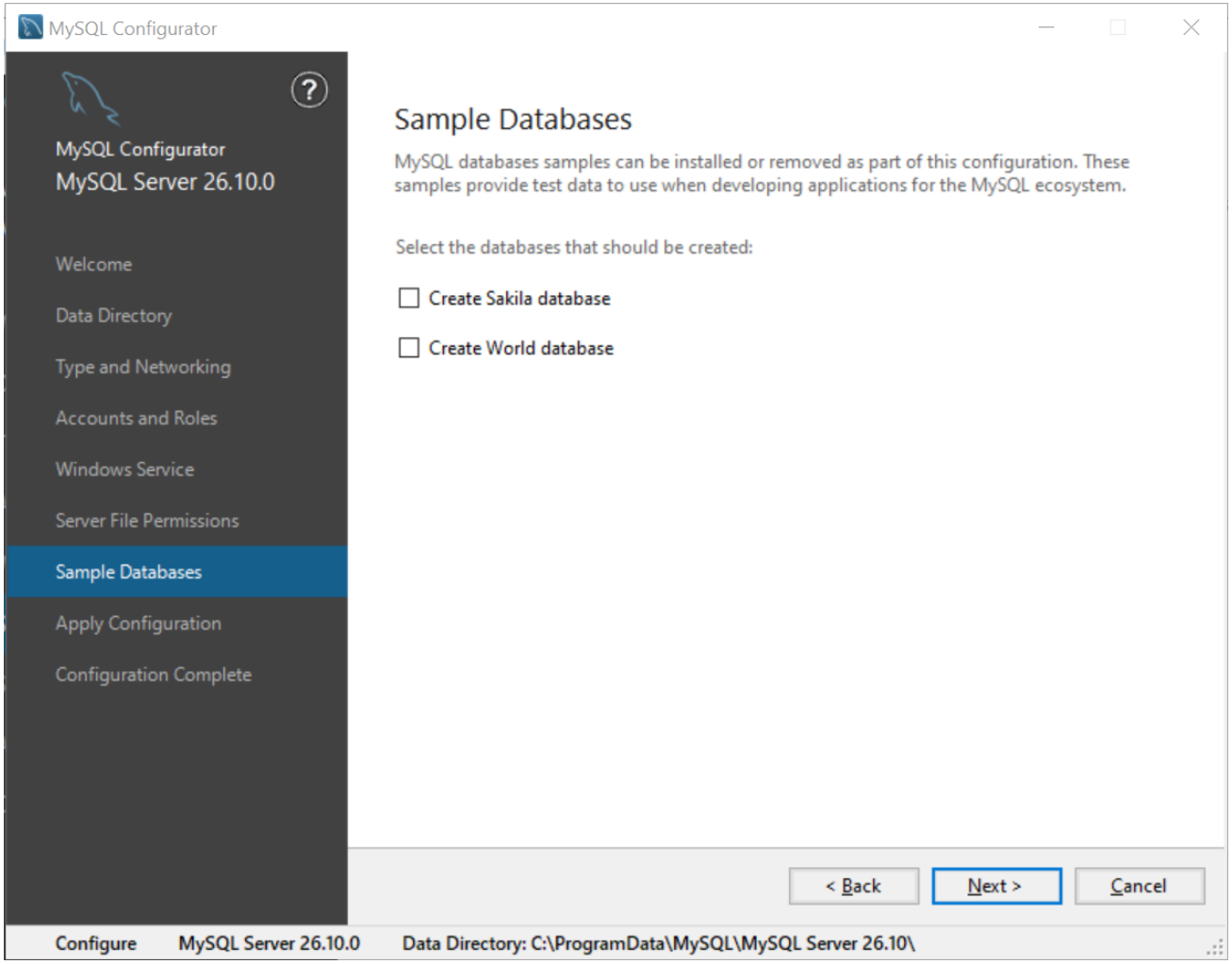
MySQL Configurator (5)



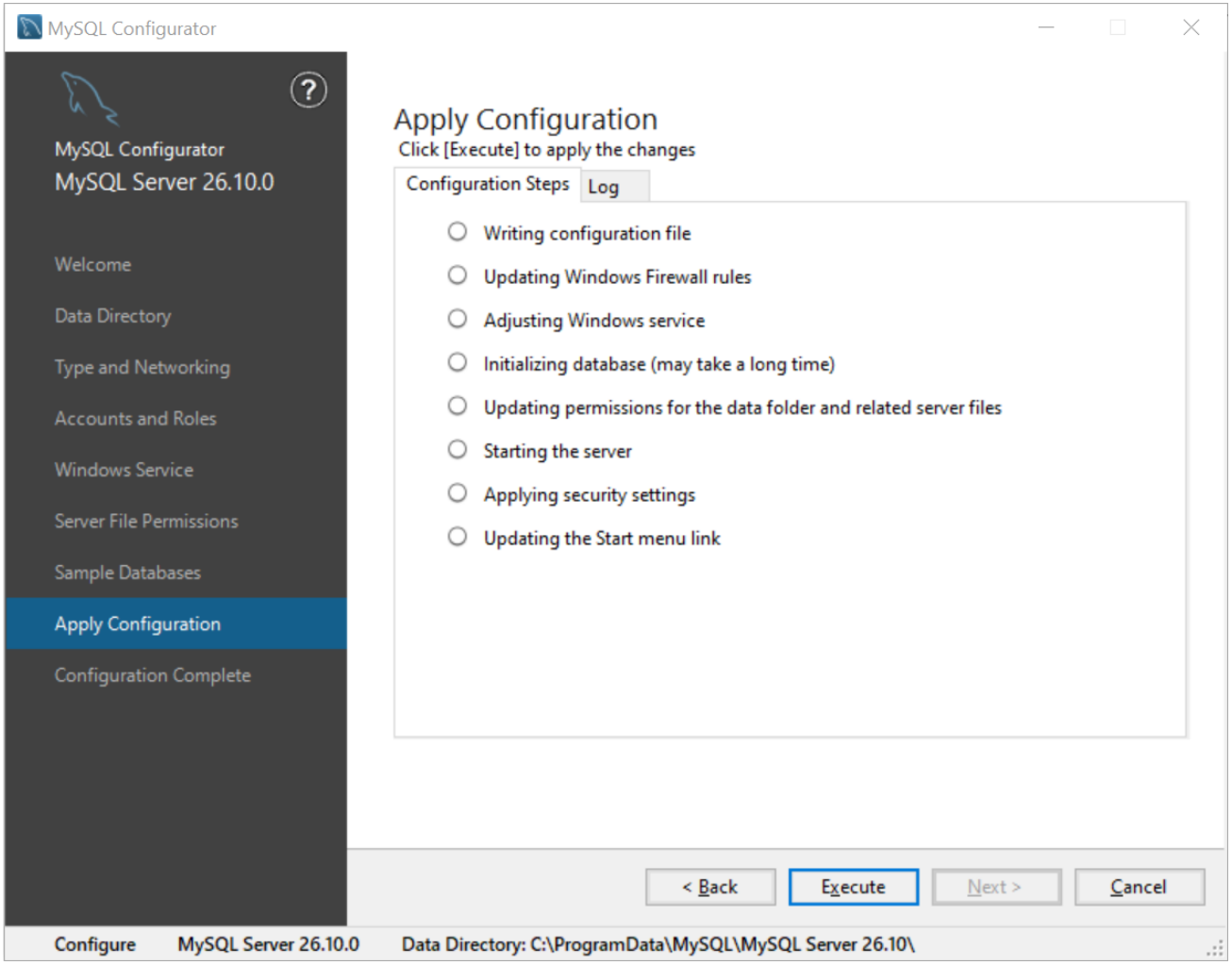
MySQL Configurator (6)



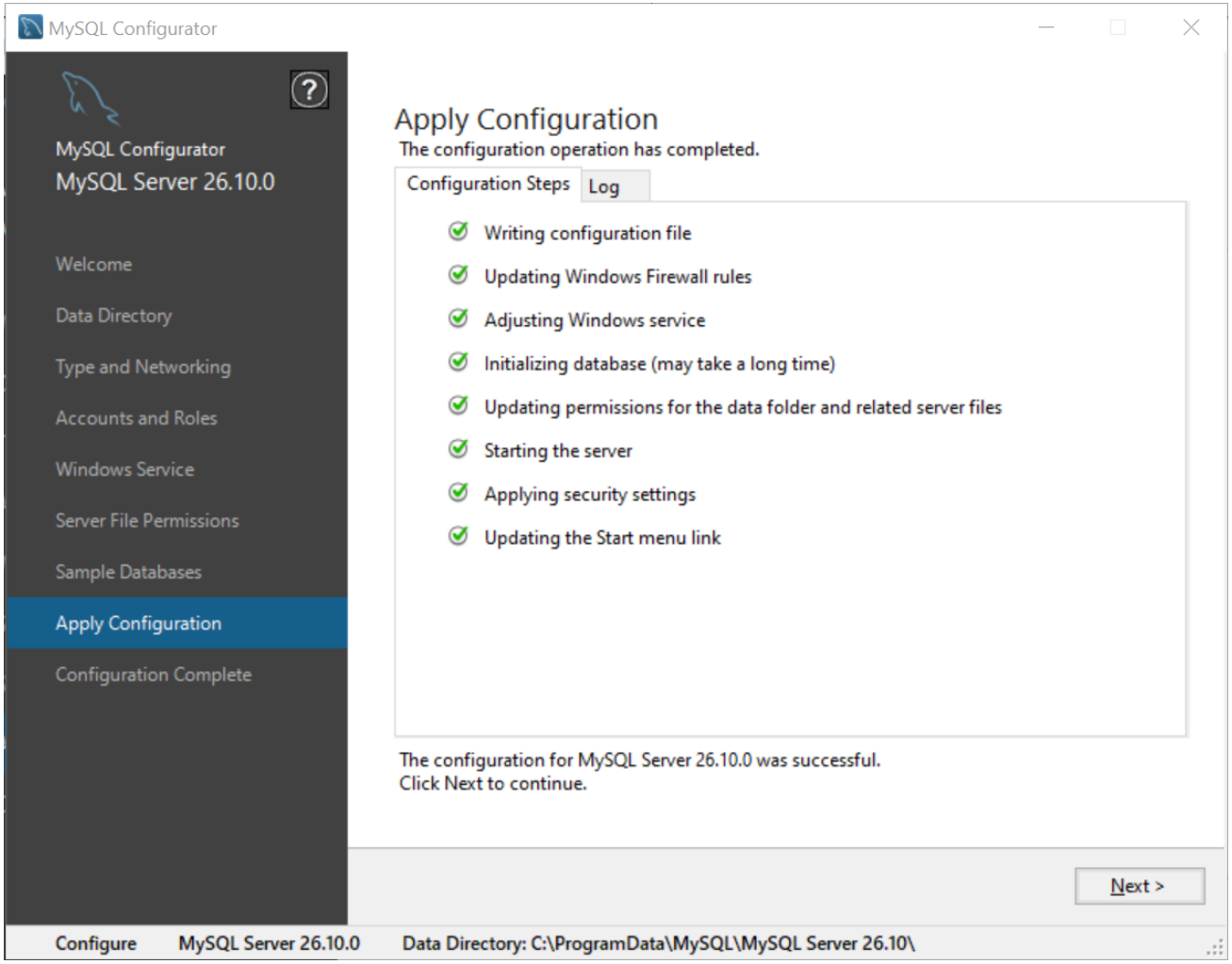
MySQL Configurator (7)



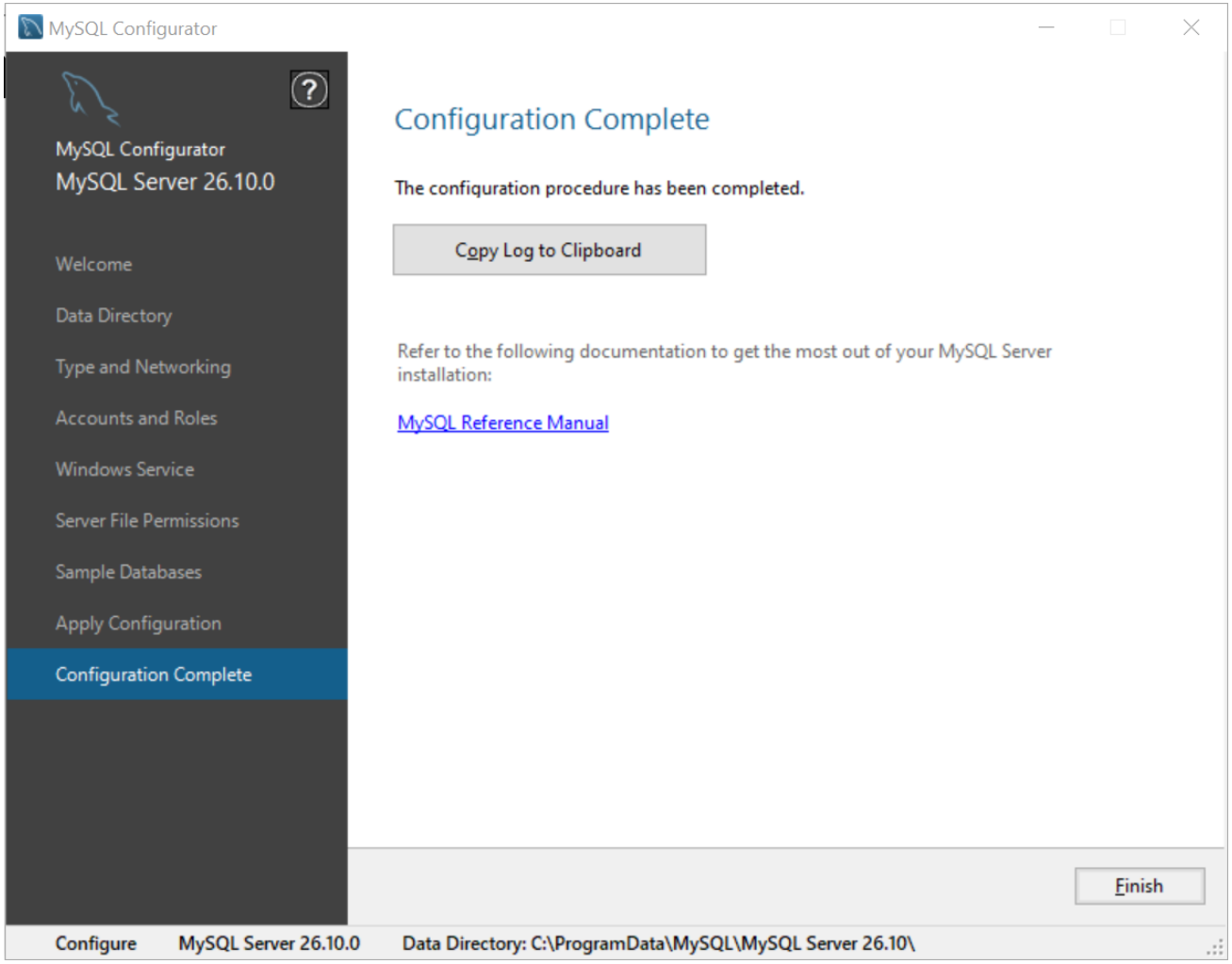
MySQL Configurator (8)



MySQL Configurator (9)



MySQL Configurator (10)



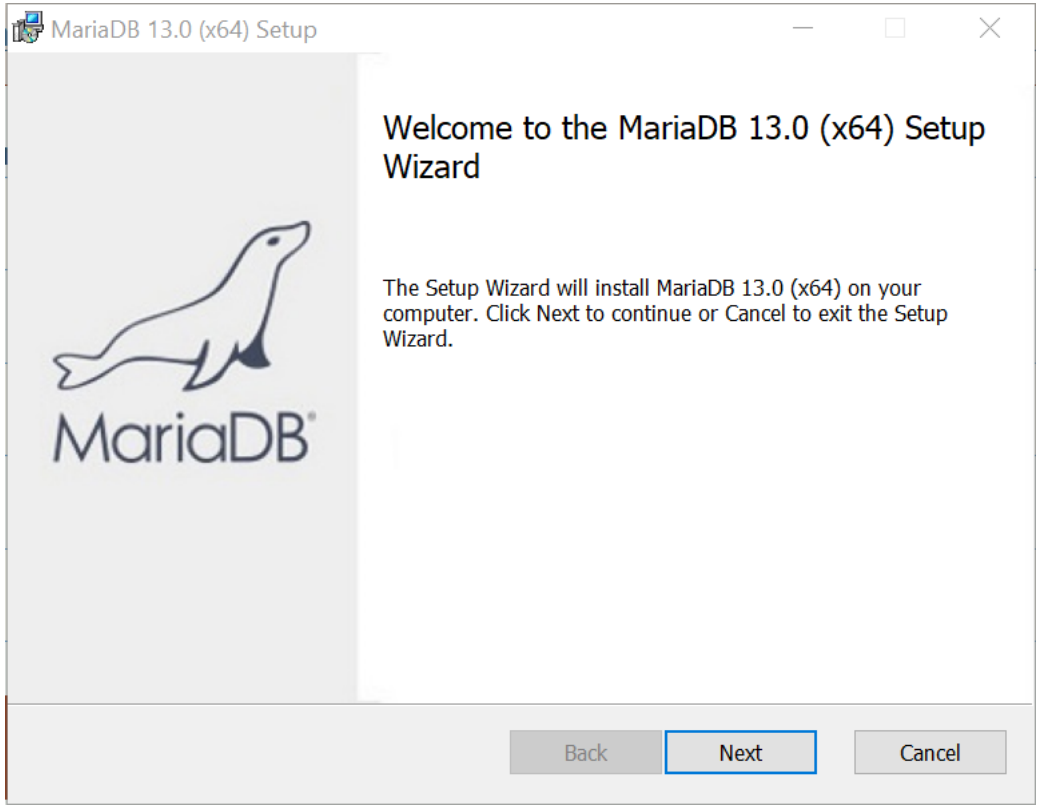
INSTALLAZIONE DI MARIADB

MariaDB

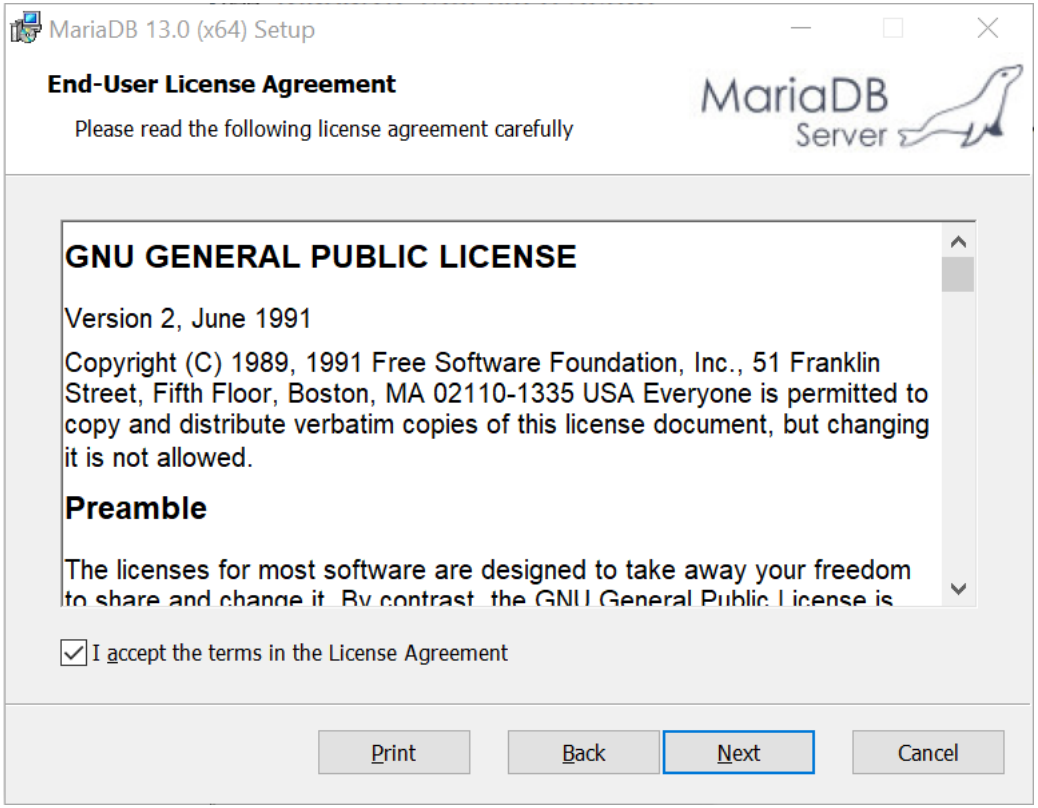


- MariaDB Server
  - Versione liberamente scaricabile del motore di database MariaDB
  - Disponibile per Windows, macOS, Linux
  - Anche per container Docker
  - Licenza GPL
- Su Windows, scarichiamo il pacchetto MSI
  - https://mariadb.org/download

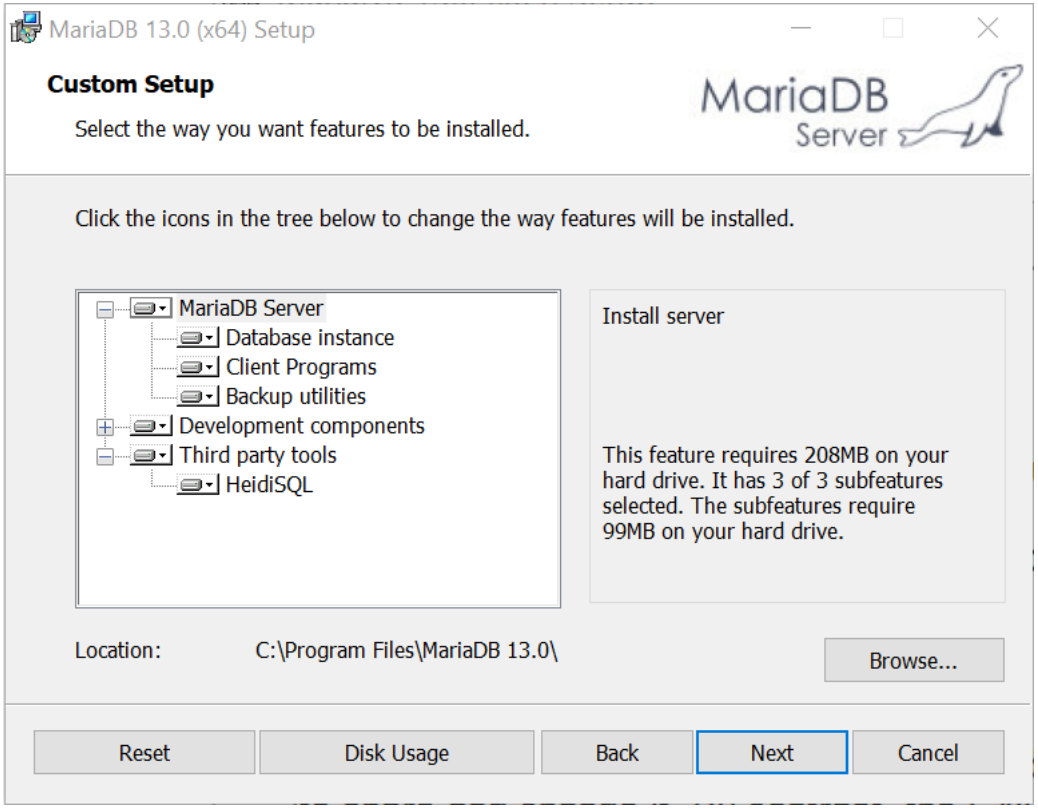
Installazione di MariaDB (1)



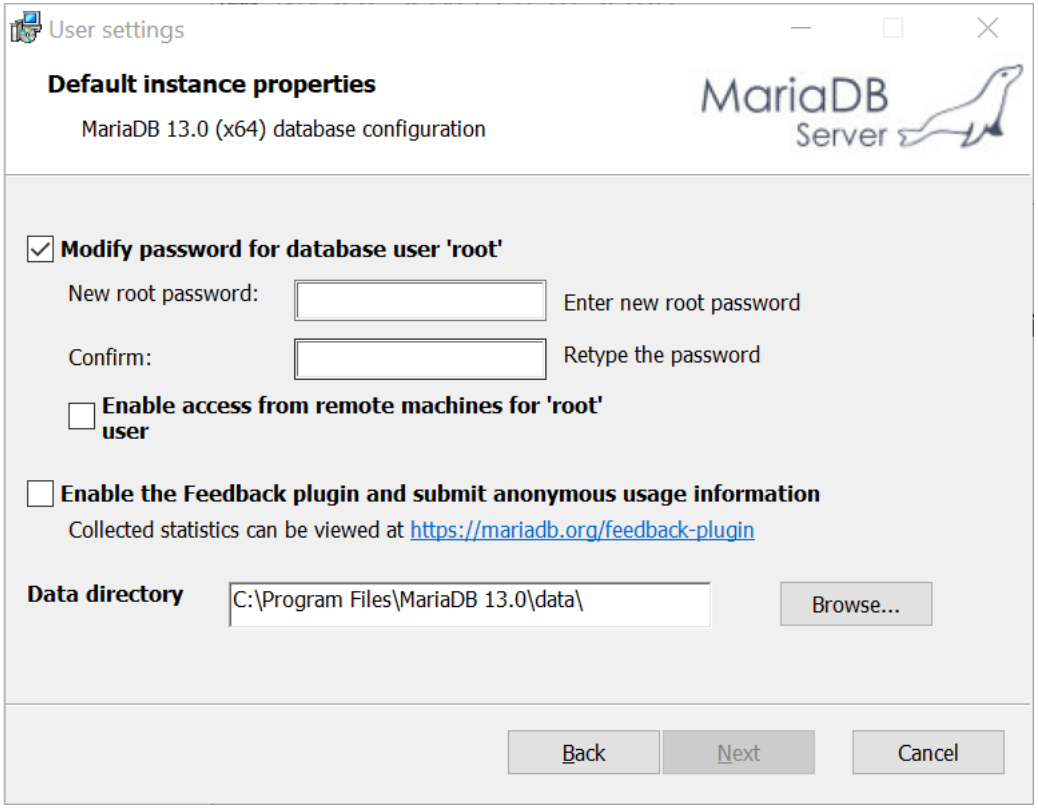
Installazione di MariaDB (2)



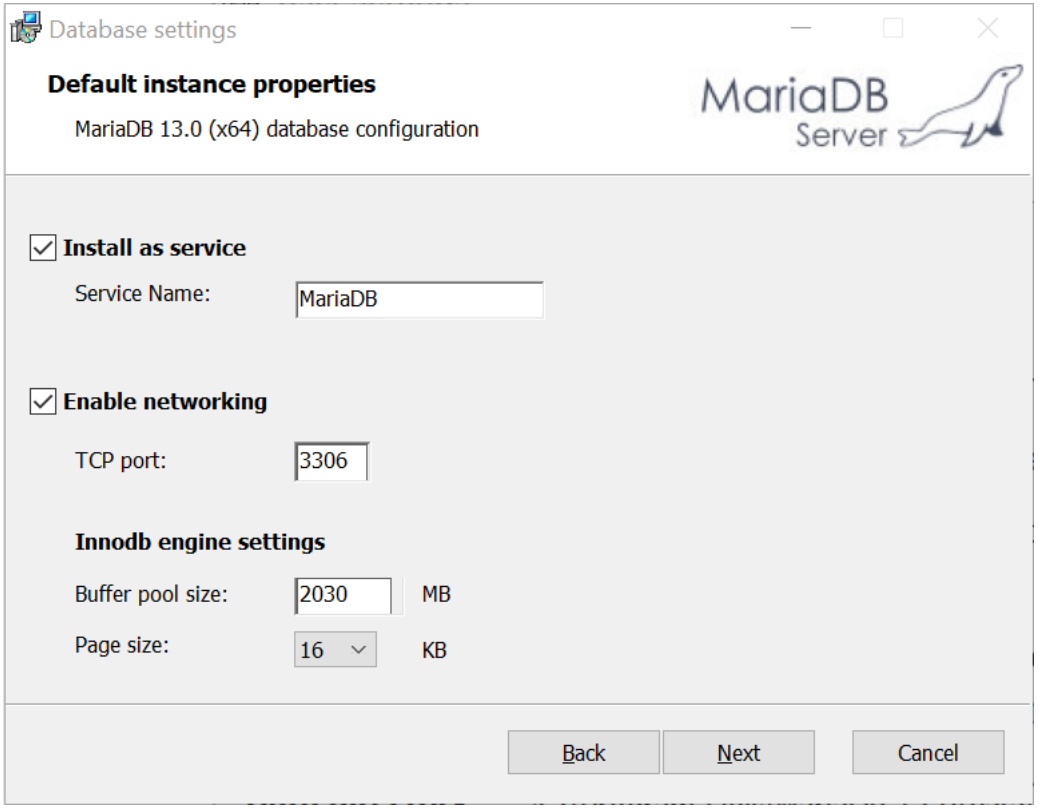
Installazione di MariaDB (3)



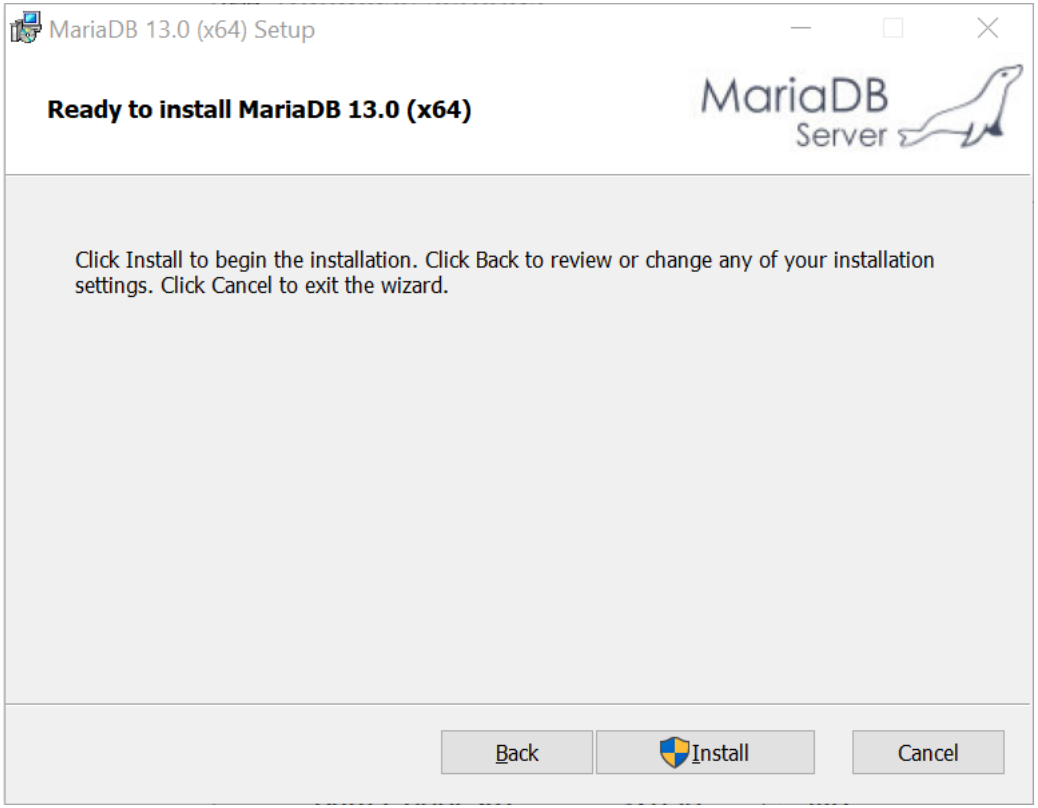
Installazione di MariaDB (4)



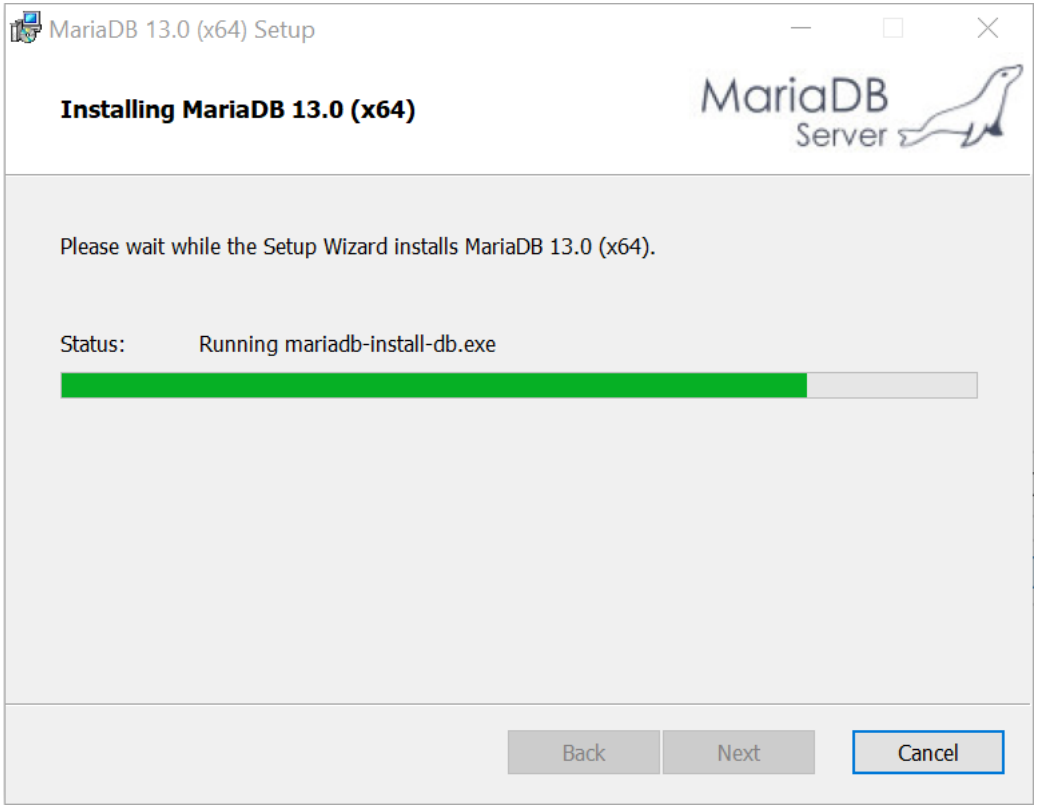
Installazione di MariaDB (5)



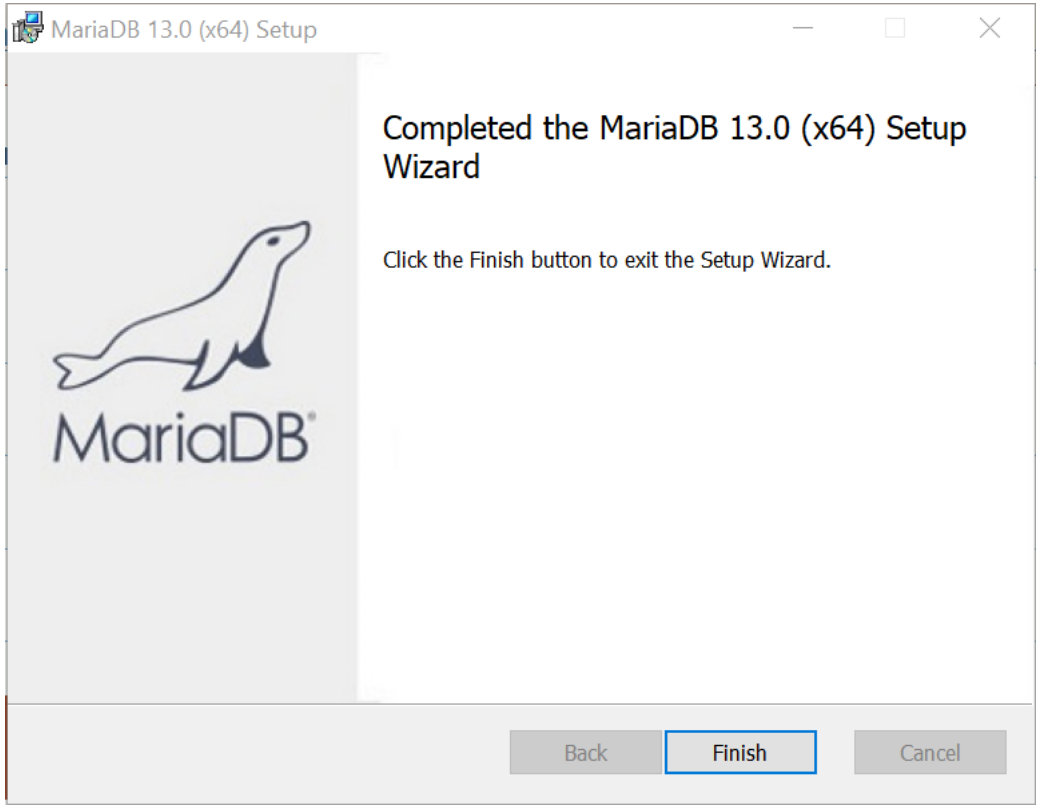
Installazione di MariaDB (6)



Installazione di MariaDB (7)



Installazione di MariaDB (8)



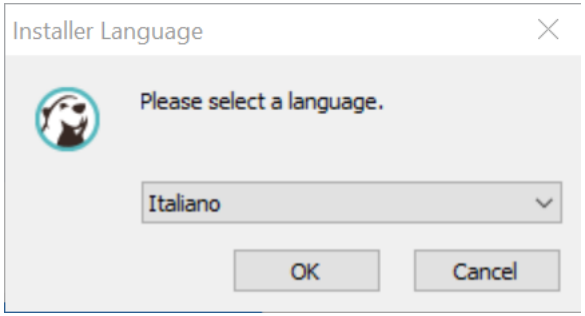
INSTALLAZIONE DI DBEAVER

DBeaver

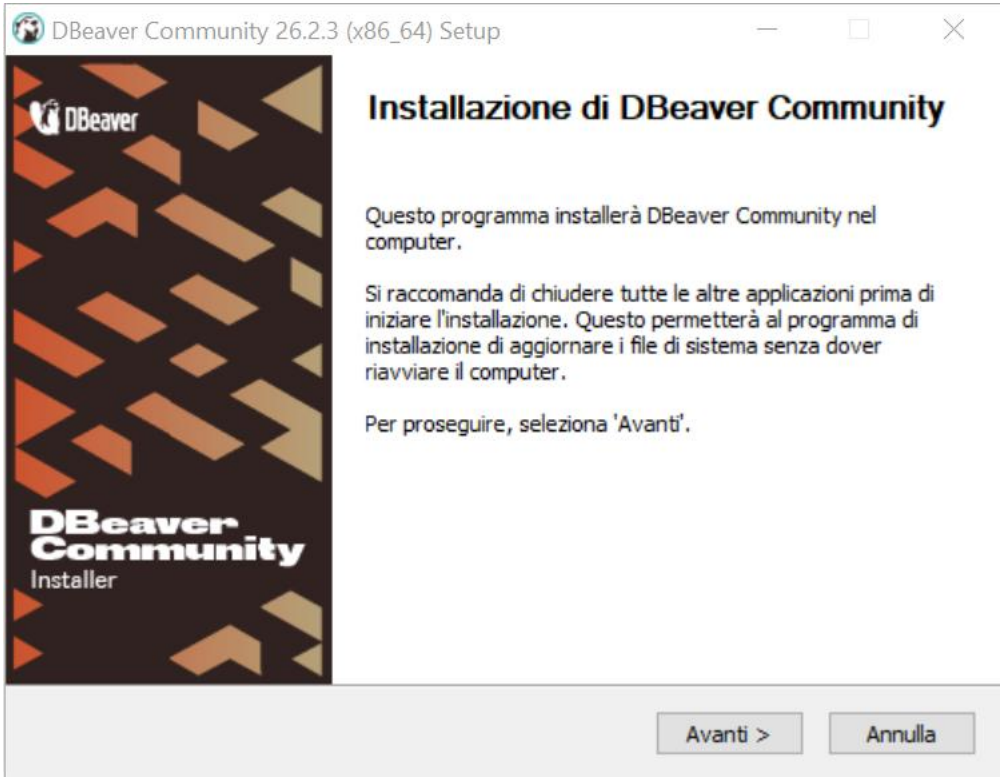


- DBeaver
  - Strumento per la gestione di database
  - Può gestire MySQL, MariaDB, PostgreSQL, SQLite, ecc…
  - Disponibile per Windows, macOS, Linux
  - Licenza GPL
- Su Windows, scarichiamo il pacchetto EXE
  - https://dbeaver.io/download

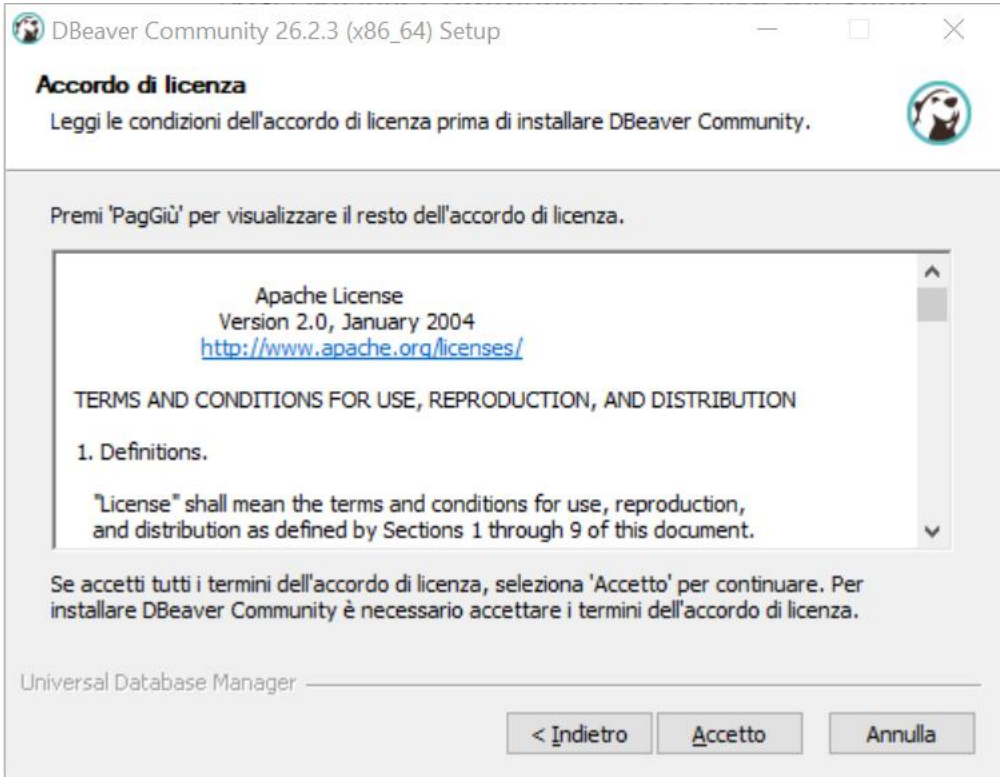
Installazione di DBeaver (1)



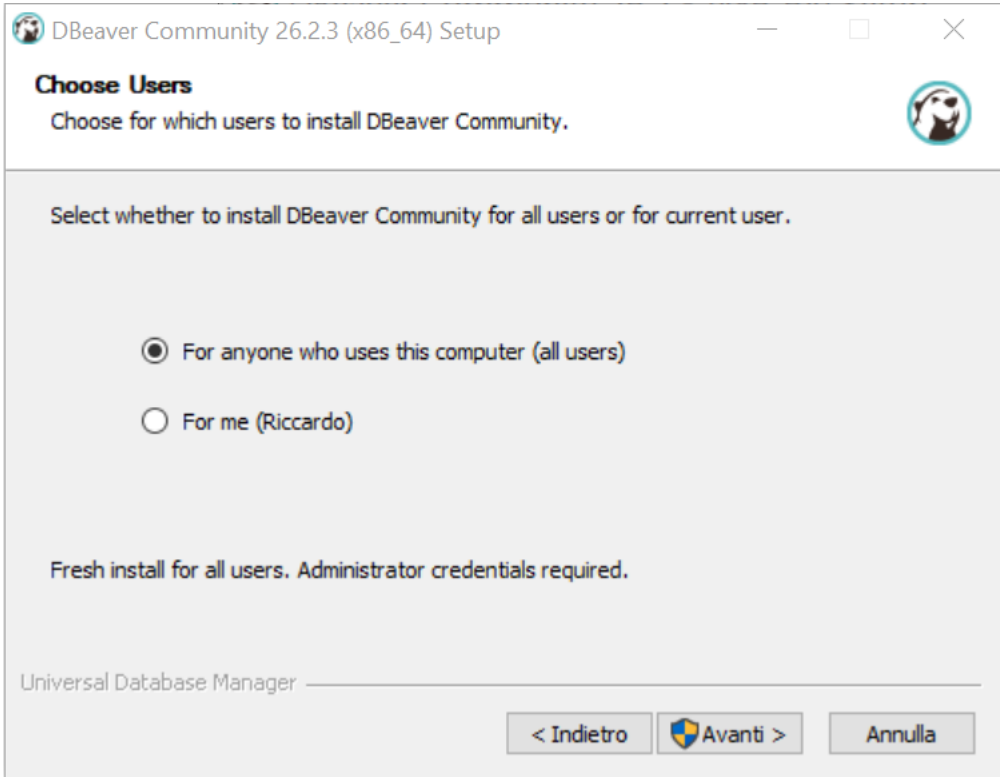
Installazione di DBeaver (2)



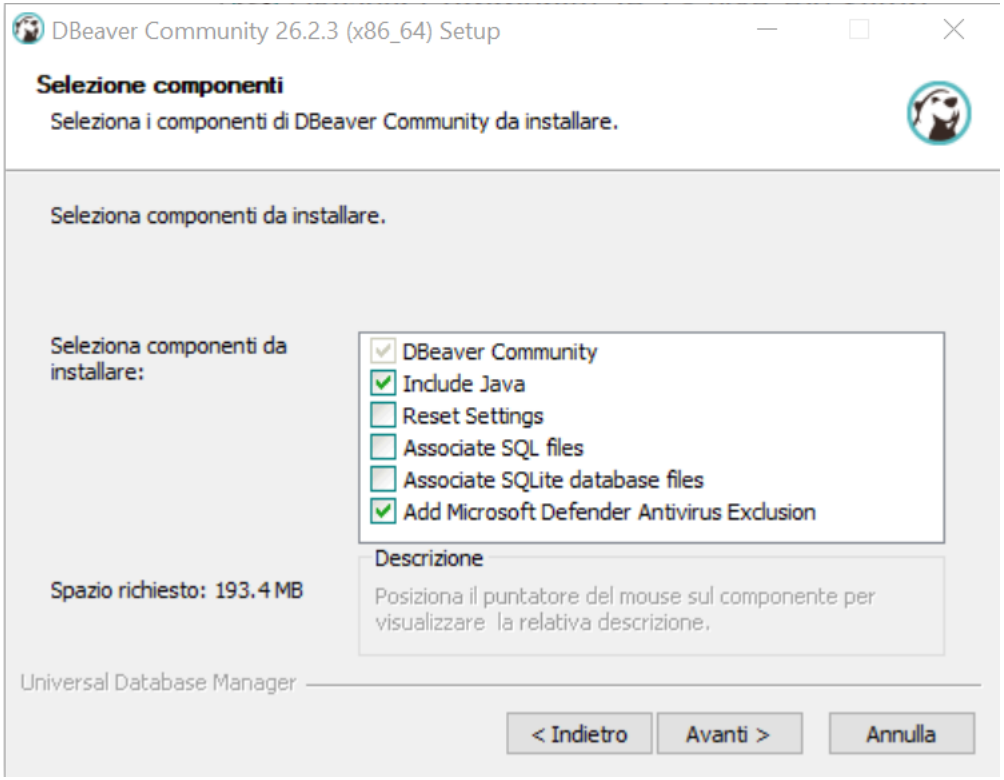
Installazione di DBeaver (3)



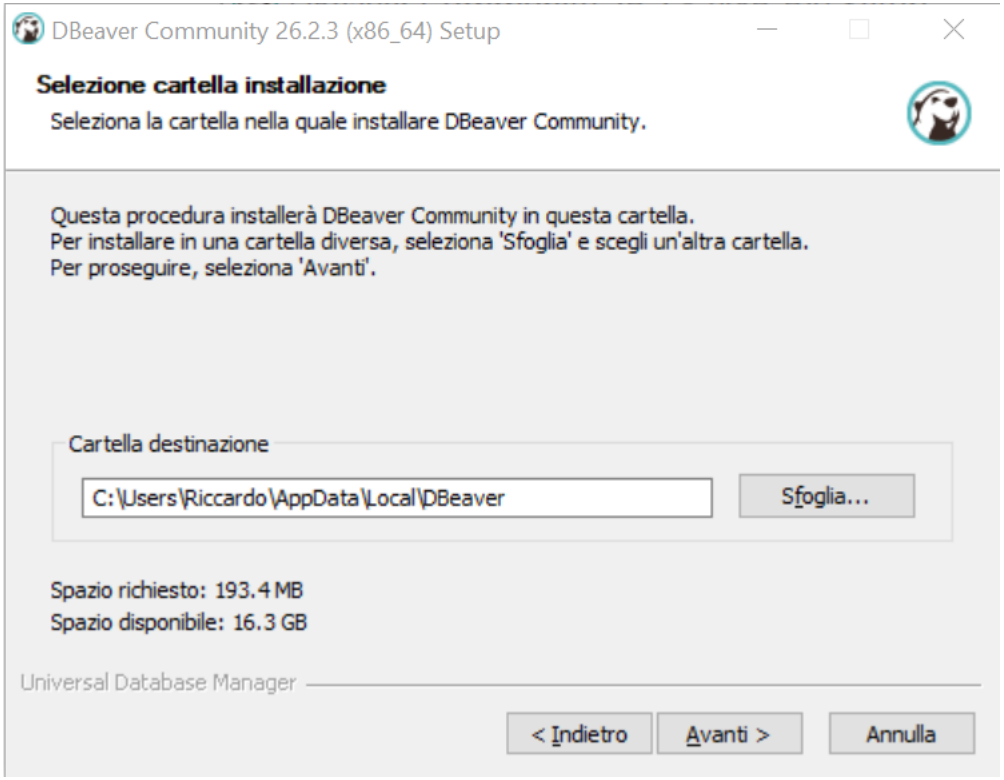
Installazione di DBeaver (4)



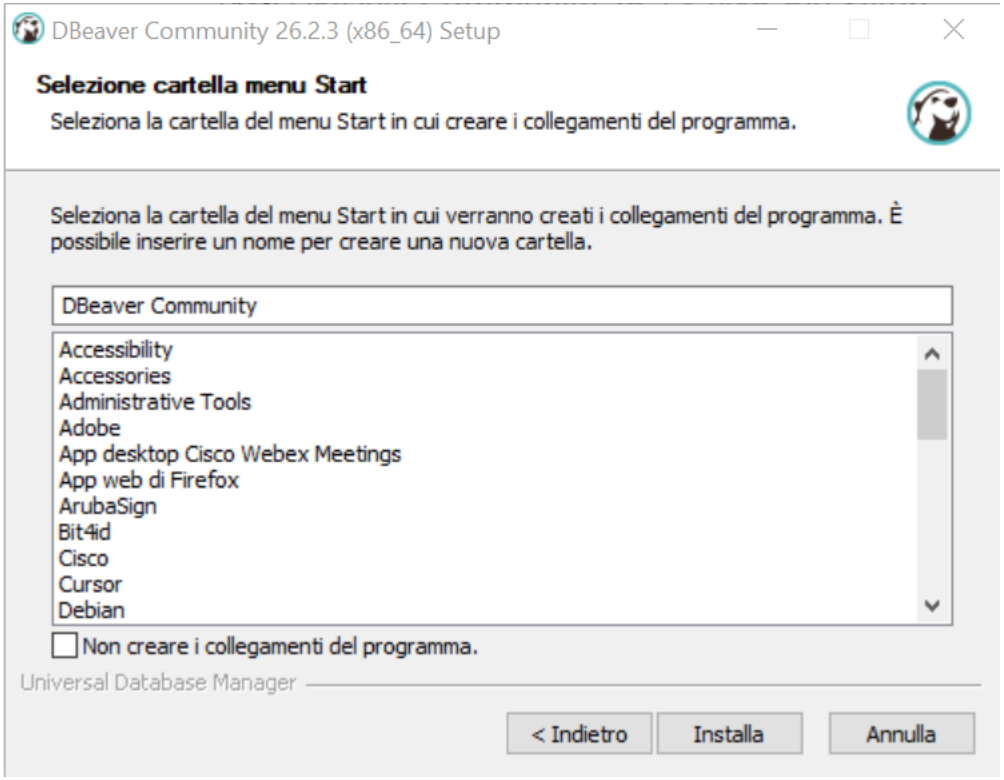
Installazione di DBeaver (5)



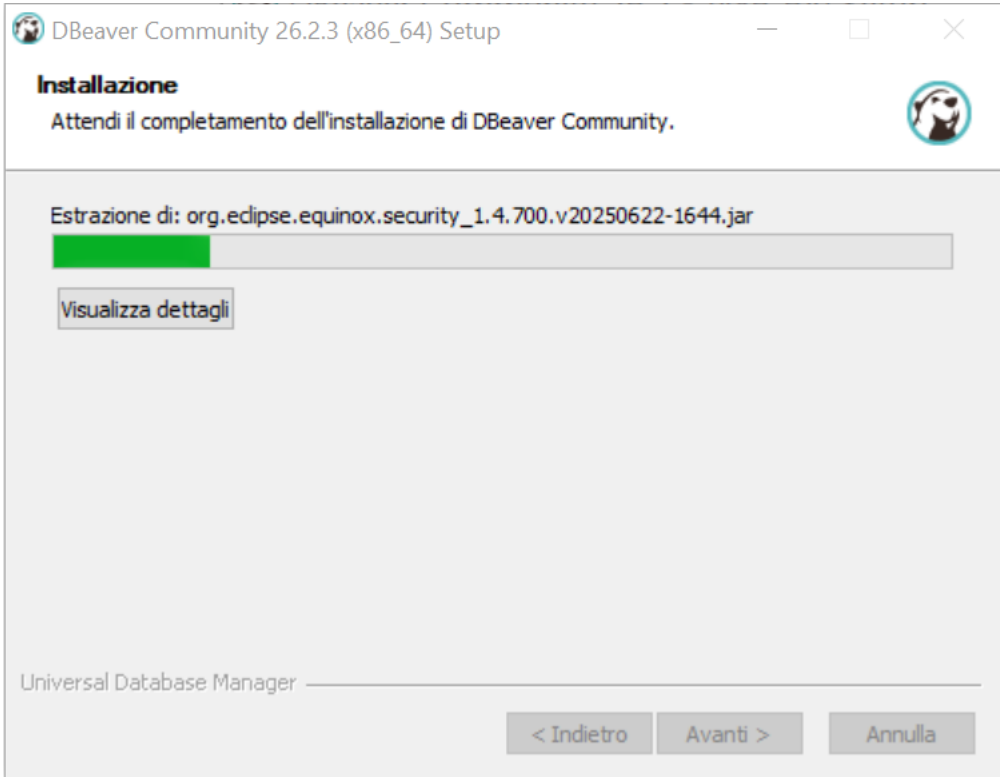
Installazione di DBeaver (6)



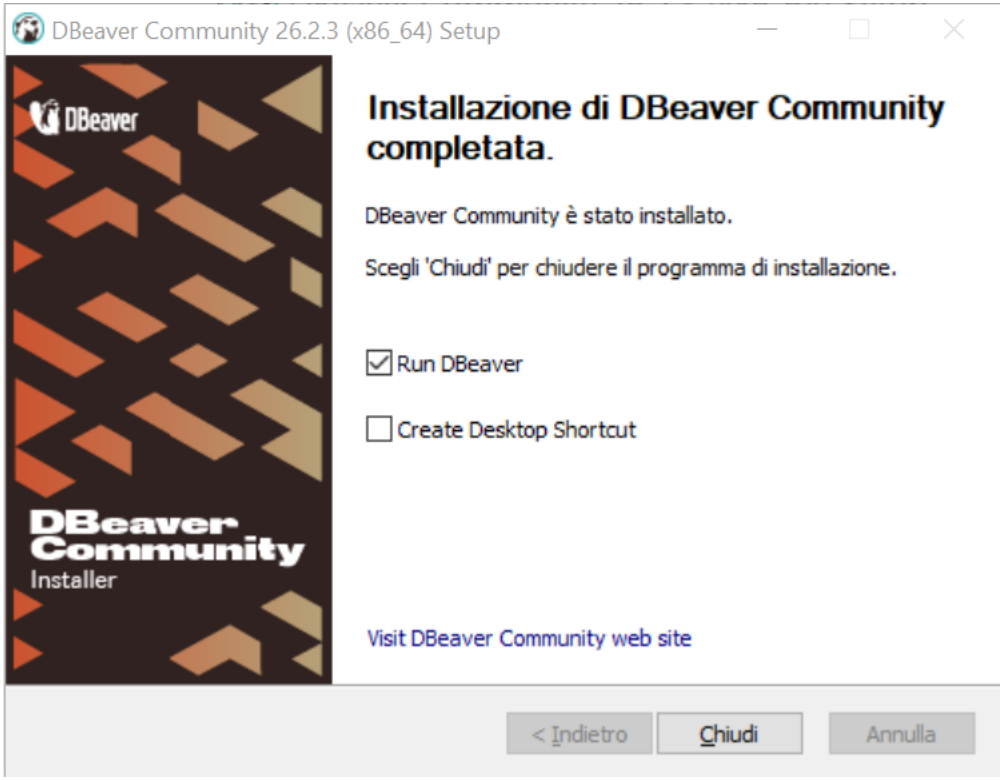
Installazione di DBeaver (7)



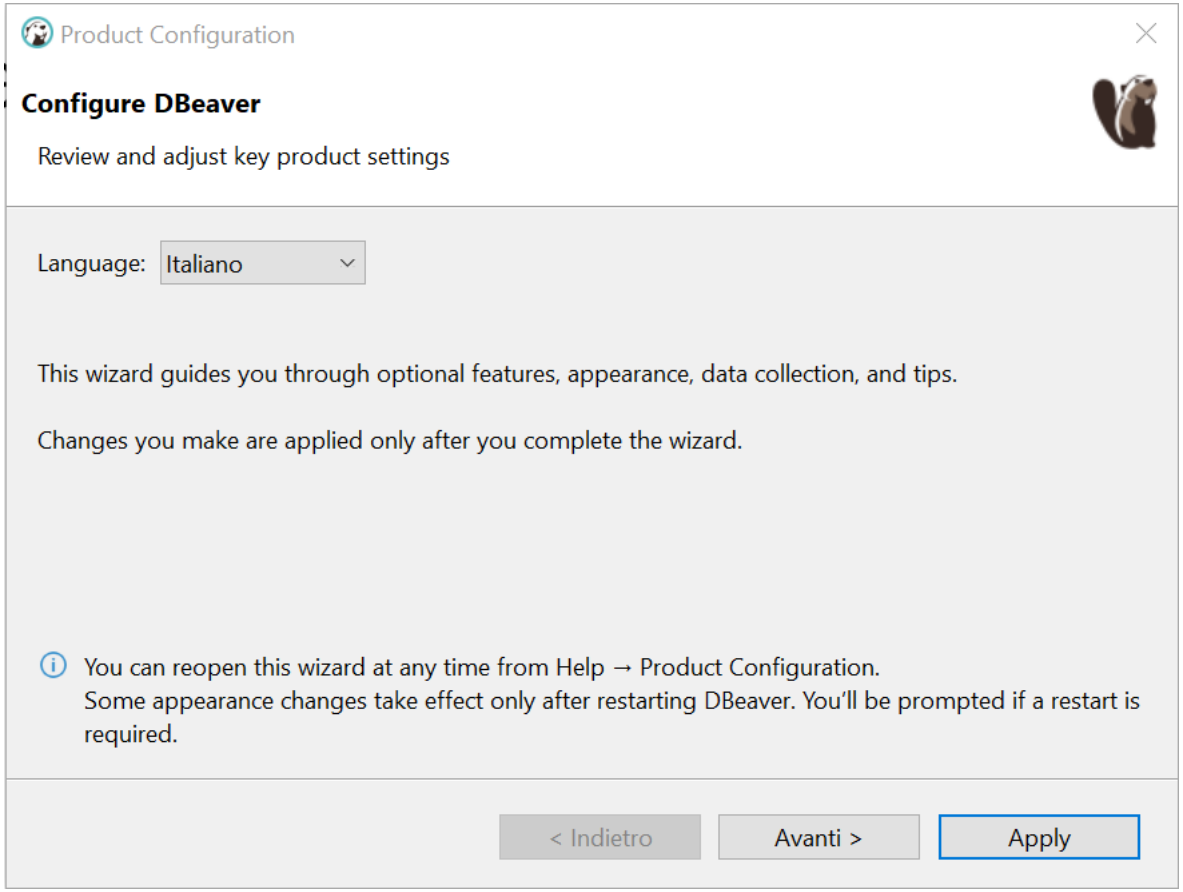
Installazione di DBeaver (8)



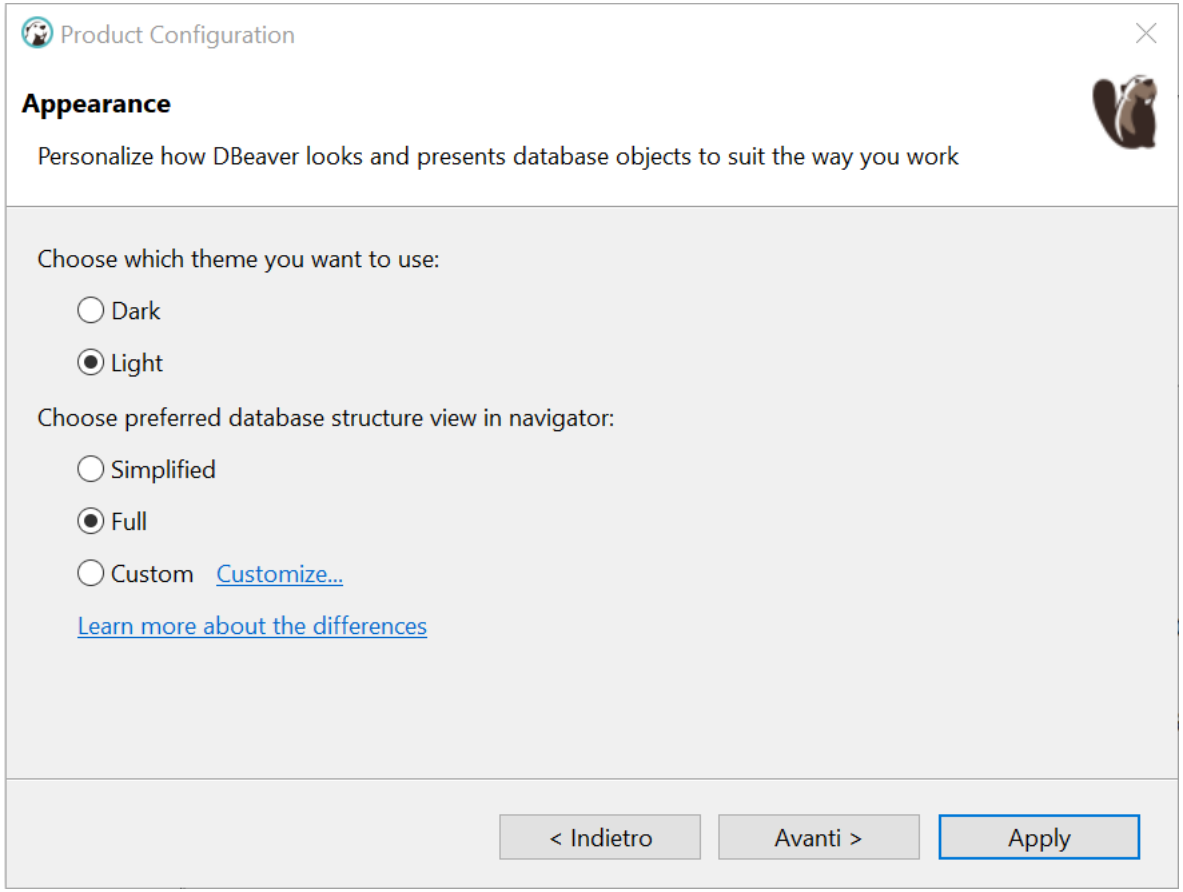
Installazione di DBeaver (9)



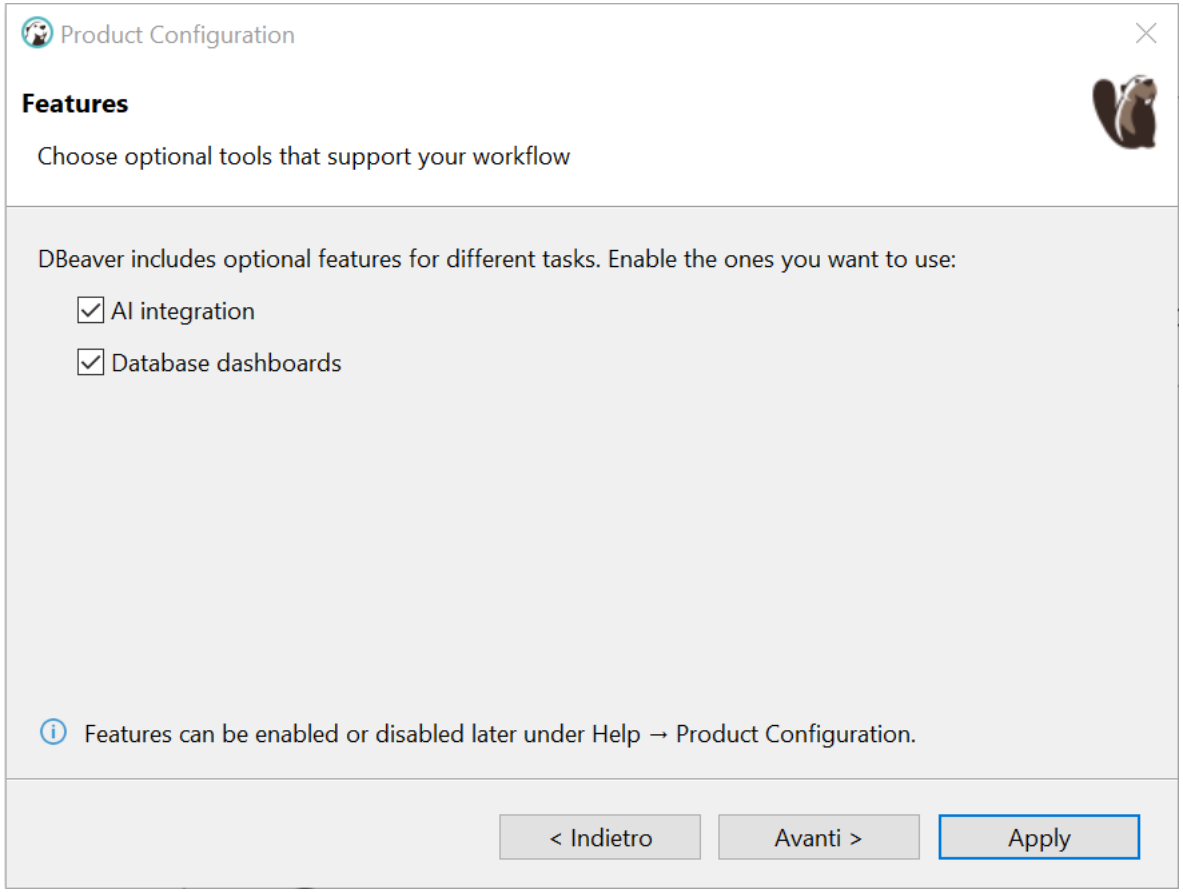
Configurazione di DBeaver (1)



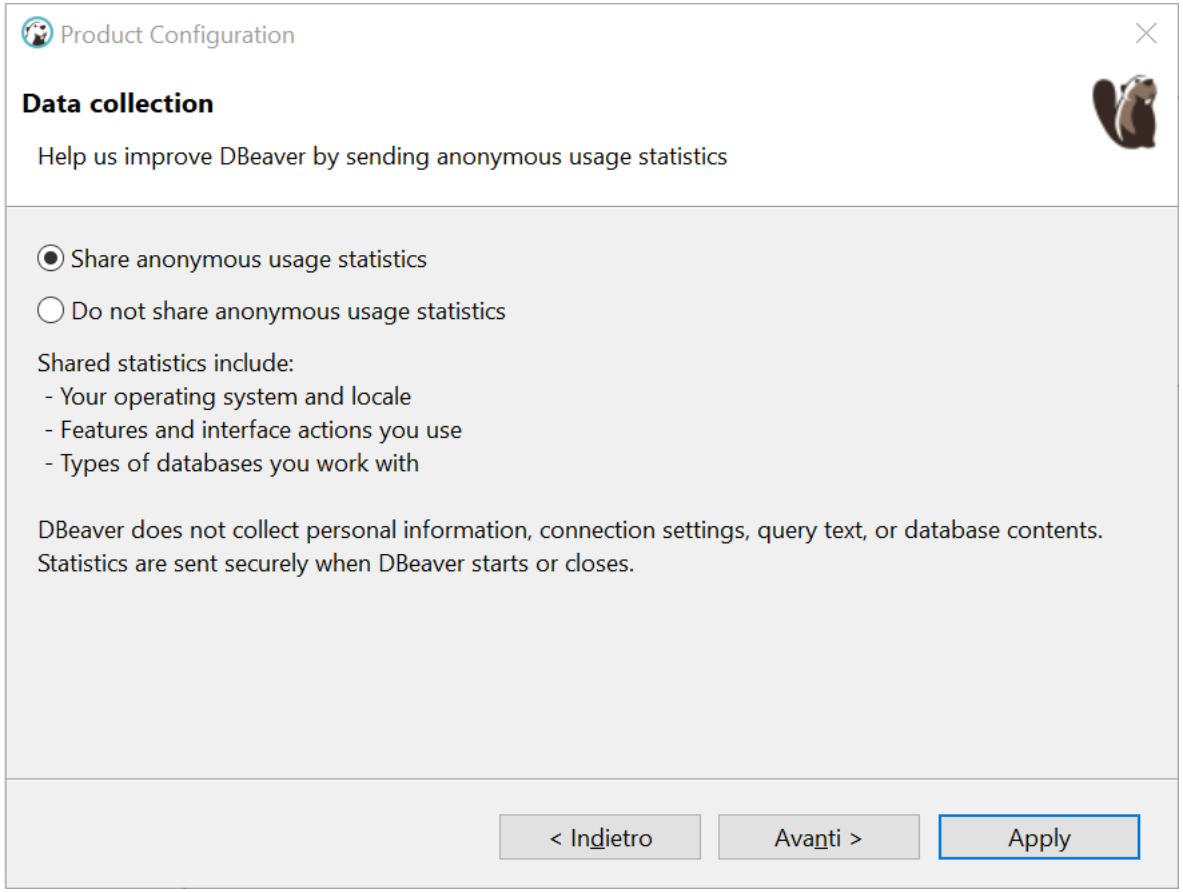
Configurazione di DBeaver (2)



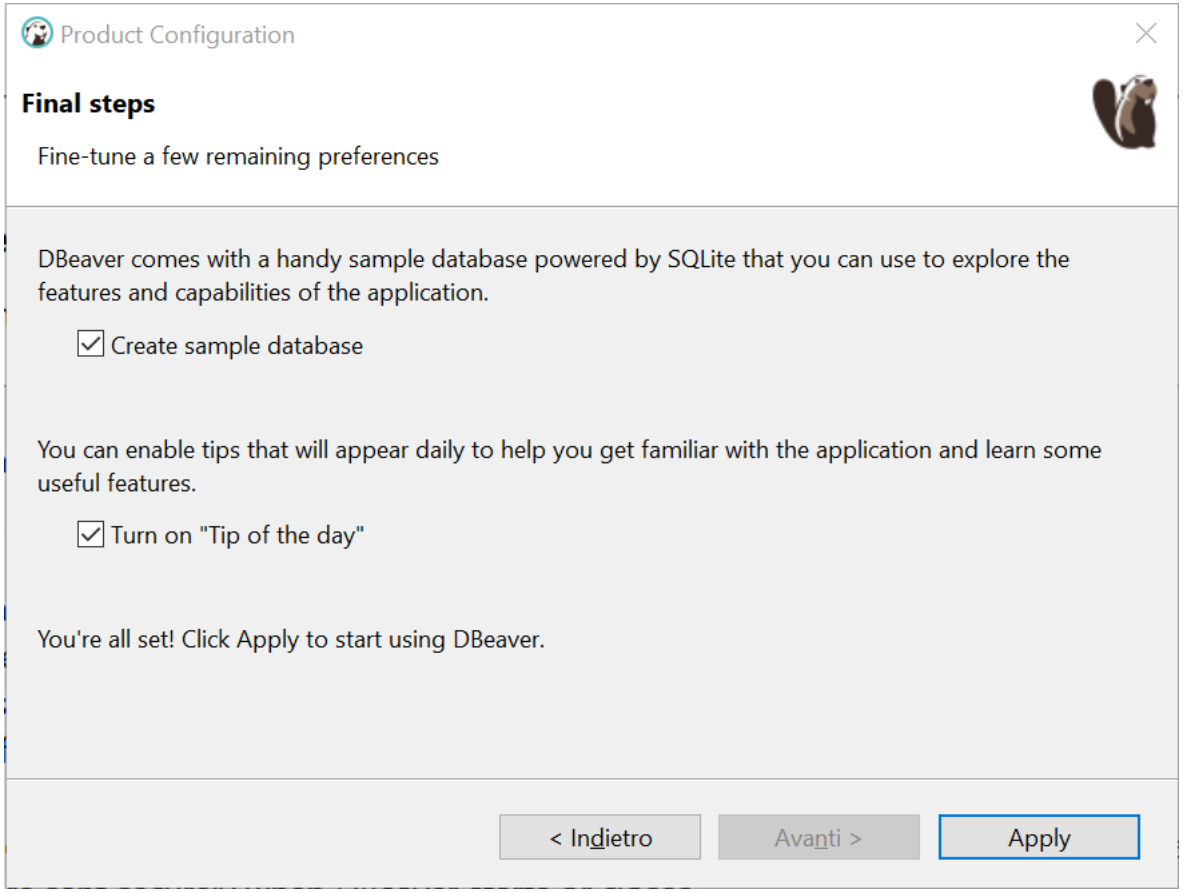
Configurazione di DBeaver (3)



Configurazione di DBeaver (4)



Configurazione di DBeaver (5)



UTILIZZO DI MYSQL/MARIADB DA RIGA DI COMANDO

Connessione ad un database

- Aprire una connessione:
  - $> mysql -h host -u user -p
  - $> mysql -u user -p se stiamo collegandoci in locale
- Vedere l'elenco dei database esistenti:
  - mysql> SHOW DATABASES;
- Creare un database:
  - mysql> CREATE DATABASE miodb;
- Utilizzare un database:
  - mysql> USE miodb

Creazione di nuovo utente

- Creazione di un nuovo utente:
  - mysql> CREATE USER 'username'@'localhost' IDENTIFIED BY 'password';
- Concedere all'utente i privilegi sul database:
  - mysql> GRANT ALL ON miodb.* TO 'username'@'localhost';

Creazione di una tabella

- Visualizzare le tabelle esistenti
  - mysql> SHOW TABLES;
- Creare una nuova tabella
  - mysql> CREATE TABLE studente (id INT NOT NULL, nome VARCHAR(45) NULL, cognome VARCHAR(45) NULL, matricola INT NULL, sesso CHAR(1) NULL, datanascita DATETIME NULL, PRIMARY KEY (id));
- Visualizzare informazioni su una tabella
  - mysql> DESCRIBE studente;

Inserimento dati in una tabella

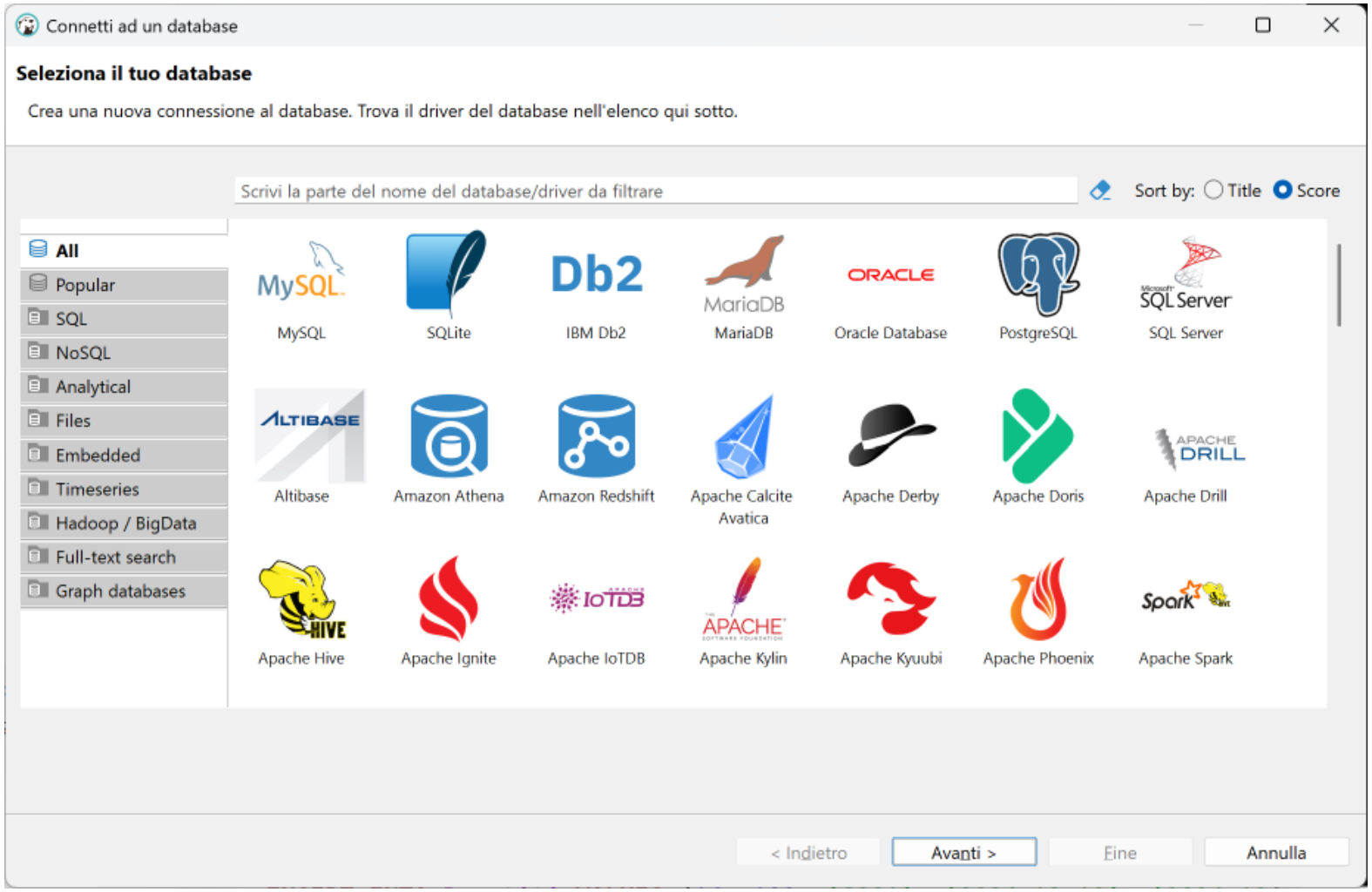
- Inserire dati in una tabella
  - mysql> INSERT INTO studente (id, nome, cognome, matricola, sesso, datanascita) VALUES ('1', 'Mario', 'Rossi', '12345', 'M', '1991-02-15');
- Inserire dati in una tabella da file
  - mysql> LOAD DATA LOCAL INFILE '/path/studente.txt' INTO TABLE studente;
- Visualizzare i dati inseriti:
  - mysql> SELECT * FROM studente;

UTILIZZO DI DBEAVER

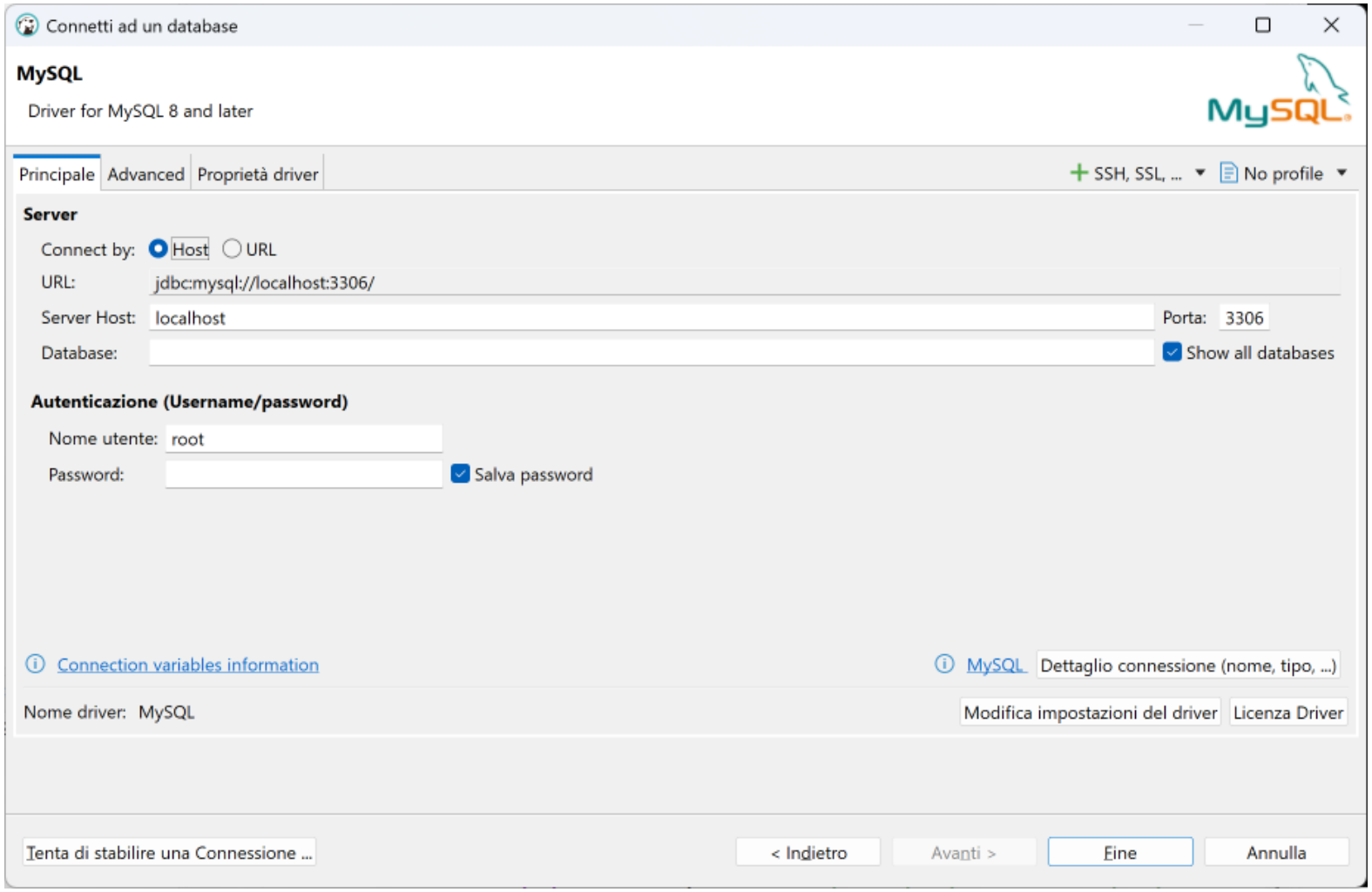
Creazione della connessione (1)

- Occorre creare la connessione al database
- Database → Nuova connessione
- Cercare MySQL o MariaDB, in base alla propria installazione, cliccare Avanti

Creazione della connessione (2)



Creazione della connessione (3)



Creazione della connessione (4)

- Per configurare il driver, andare in Proprietà del driver, scegliere Modifica Driver
- Nella finestra che si aprirà scegliere Librerie
- Nella scheda librerie premere su Aggiungi File e aggiungere il file .jar scaricabile da:
  - https://dev.mysql.com/downloads/connector/j/
    - Scaricare il file zip e cercare il .jar nella cartella principale
  - https://mariadb.com/downloads/connectors/
    - Scarica solo il file jar

Creazione di un database

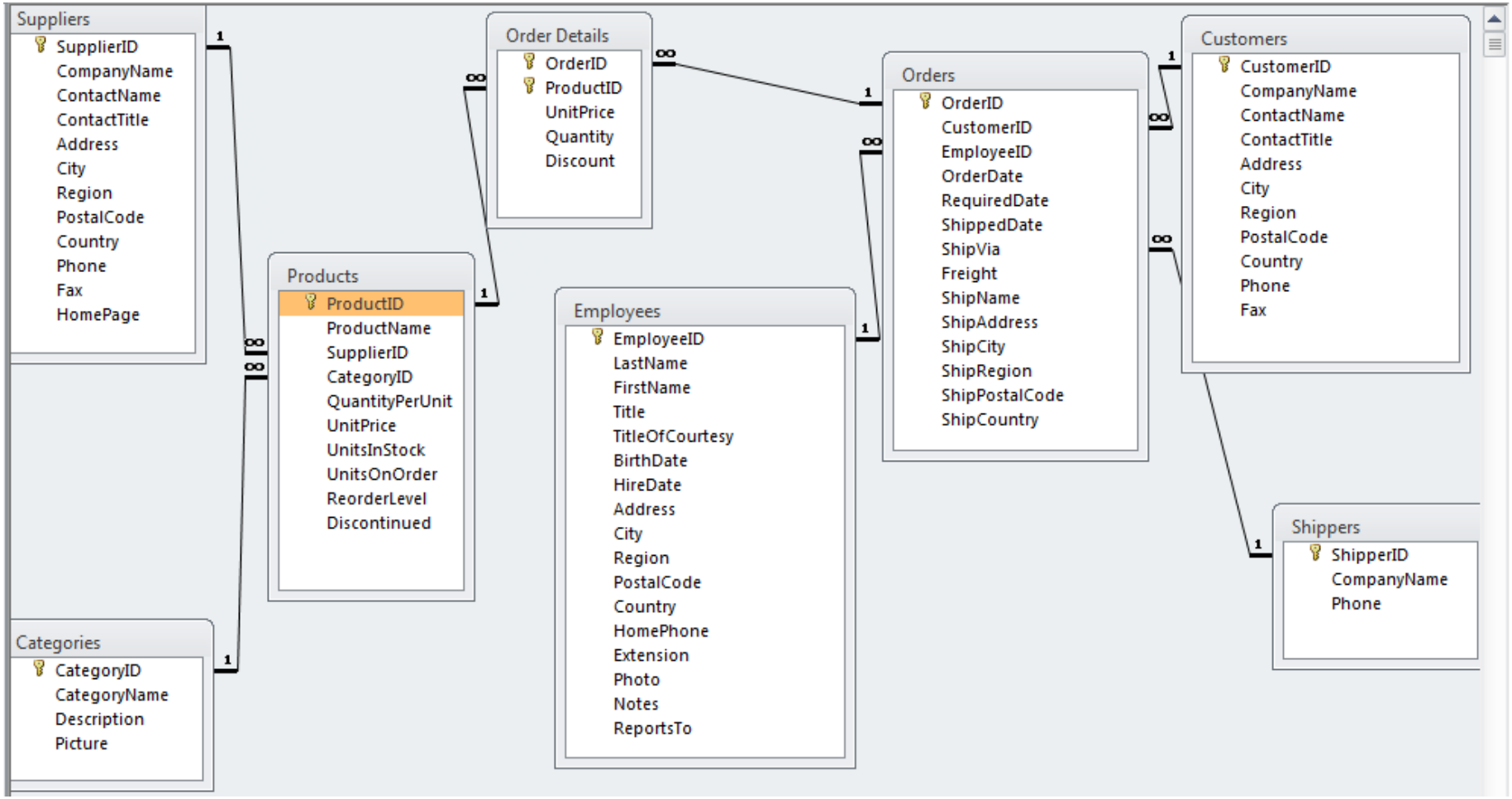
- Selezionare la connessione appena creata
- Clic destro → Crea → Database
  - Inserire il nome scelto per il database

- OPPURE…
- Selezionare la connessione appena creata
- Clic destro → Editor SQL → Nuovo Script SQL
- Inserire: CREATE DATABASE northwind;
- ALT+X

Esecuzione di uno script

- Carichiamo lo script del database Northwind
  - Editor SQL → Carica script SQL
  - Clic destro sullo script → Esegui → Imposta connessione attiva
  - Clic destro sullo script → Esegui → Esegui Script SQL
  - OPPURE: premere ALT + X
- Lo script è presente sulla pagina Moodle dell'insegnamento

Il database Northwind



JDBC E MYSQL/MARIADB

JDBC

- Java Database Connectivity (JDBC) è una interfaccia per la programmazione di applicazioni (API) per il linguaggio di programmazione Java
- Definisce come un client può accedere ad un database
- È parte della piattaforma Java Standard Edition
- Fornisce metodi per eseguire query e aggiornare i dati in un database
- È orientata verso i database relazionali

Connessione a database remoto

- Con JDBC possiamo connetterci anche ad un database remoto, con un URL nella forma: jdbc:mysql://host:port/database, dove:
  - host rappresenta l'host utilizzato, localhost di default
  - port è la porta utilizzata, 3306 di default
  - database è il nome del database

Connector/J

- Un driver JDBC open source, che consente di connettersi ad un database MySQL utilizzando Java
- Il codice Java è indipendente dal database utilizzato, grazie all'uso di JDBC
- Il driver si registra automaticamente presso il *DriverManager*: non serve *Class.forName()*
- Sito: https://dev.mysql.com/downloads/connector/j/

Configurazione all'uso di Connector/J

- Il file mysql-connector-j-26.7.0.jar deve essere incluso all'interno del classpath per poter essere utilizzato
  - java -cp .:/percorso/mysql-connector-j-26.7.0.jar JdbcDemo
  - (su Windows il separatore del classpath è ; anziché :)
- In alternativa, con Maven: dipendenza *com.mysql:mysql-connector-j:26.7.0*
- Nel caso di utilizzo di una IDE dobbiamo configurarla in modo opportuno, per indicare il percorso della libreria

JDBC Connection (1)

- Prima di tutto occorre inserire l'import seguente:
  import java.sql.*;

- Dobbiamo quindi stabilire una connessione al database:
  Connection db = DriverManager.getConnection(url, username, password);
- Dato che il driver supporta un alto numero di proprietà aggiuntive, possiamo utilizzare un oggetto Properties

  Properties props = new Properties ();

JDBC Connection (2)

```
import java.sql.*;
import java.util.Properties;
public class JdbcDemo {
    public static void main(String[] args) throws SQLException {
        String url = "jdbc:mysql://%INDIRIZZO%/%DATABASE%";
        Properties props = new Properties ();
        props.setProperty("user", "%NOME_UTENTE%");
        props.setProperty("password", "%PASSWORD%");
        Connection conn = DriverManager.getConnection(url, props);
        System.out.println("Connessione funzionante");
        conn.close();
    }
}
```

JDBC Query

- Per effettuare una query occorre utilizzare uno Statement o un PreparedStatement
- Si utilizzerà una comune query SQL, rispettandone la sintassi

Creare una tabella

```
Statement stmt = conn.createStatement();
String sql = "CREATE TABLE student "
      + "(id INTEGER not NULL, "
      + " name VARCHAR(255), "
      + " surname VARCHAR(255), "
      + " birthplace VARCHAR(255), "
      + " PRIMARY KEY ( id ))";
stmt.executeUpdate(sql);
System.out.println("Tabella creata!");
```

Inserire valori con Statement

```
String query = "INSERT INTO student (id, name, surname,
birthplace) VALUES (154247, 'Mario', 'Neri', 'Bolzano') ";

Statement st = conn.createStatement();

st.executeUpdate(query);

st.close();

System.out.println("Inserimento effettuato!");
```

Inserire valori con PreparedStatement

```
String query = "INSERT INTO student
(id, name, surname, birthplace) VALUES (?, ?, ?, ?) ";
PreparedStatement st = conn.prepareStatement(query);
st.setObject(1, 128853);
st.setObject(2, "Francesco");
st.setObject(3, "Neri");
st.setObject(4, "Rovereto");
st.executeUpdate();
st.close();
```

ResultSet

- La query di interrogazione fornirà come risultato un ResultSet contenente l'intero insieme dei risultati, che possiamo scorrere con un cursore
- Per analizzare un ResultSet possiamo utilizzare metodi quali *getString*(), *getInt*(), *getFloat*(), *getDate*(), *getBoolean*()

- ResultSet rs = …
- String surname = rs.getString("surname");

JDBC Query con Statement

```
Statement st = conn.createStatement();
ResultSet rs = st.executeQuery(
        "SELECT * FROM student");
while (rs.next()) {
    System.out.print("ID: " + rs.getInt(1));
    System.out.print(" - Nome: " + rs.getString("name"));
    System.out.print(" - Cognome: " + rs.getString(3));
    System.out.print(" - Luogo di nascita: " + rs.getString(4));
    System.out.println();
    }
rs.close();
st.close();
```

JDBC Query con PreparedStatement

```
String surname = "Neri";
PreparedStatement pst = conn.prepareStatement(
    "SELECT * FROM student WHERE surname = ?");
pst.setString(1, surname);
ResultSet rs1 = pst.executeQuery();
while (rs1.next()) {
    System.out.print("ID: " + rs1.getInt(1));
    System.out.print(" - Nome: " + rs1.getString("name"));
    System.out.print(" - Cognome: " + rs1.getString(3));
    System.out.print(" - Luogo di nascita: " + rs1.getString(4));
    System.out.println();
}
```

Aggiornare il database

```
String surname = "Bianchi";
PreparedStatement pst = conn.prepareStatement(
        "DELETE FROM student WHERE surname = ?");
pst.setString(1, surname);
int rowsDeleted = pst.executeUpdate();
System.out.println(rowsDeleted + " righe eliminate");
pst.close();
```

PYTHON E MYSQL/MARIADB

Python e MySQL

- Anche con Python possiamo connetterci a MySQL ed eseguire query SQL, come abbiamo fatto con Java e JDBC
- Python definisce una API standard per l'accesso ai database relazionali: la DB-API (PEP 249), implementata dai driver dei diversi database
- Il codice è così in gran parte indipendente dal database utilizzato
- Useremo il driver ufficiale di Oracle: MySQL Connector/Python
  - Esistono anche driver di terze parti (ad es. PyMySQL, mysqlclient), anch'essi conformi alla DB-API

MySQL Connector/Python

- Il driver Python ufficiale per MySQL, open source, scritto interamente in Python
- Ultima versione: 26.7.0 (luglio 2026); richiede Python 3.10 o superiore
- Installazione, preferibilmente in un ambiente virtuale (python -m venv .venv):
  - pip install mysql-connector-python
- Sito: https://pypi.org/project/mysql-connector-python/

Python Connection

- Parametri: host, database, user, password (la porta è 3306 di default)

```
import mysql.connector

conn = mysql.connector.connect(
        host="%INDIRIZZO%",
        database="%DATABASE%",
        user="%UTENTE%",
        password="%PASSWORD%",
)
print("Connessione funzionante")
conn.close()
```

Errori, chiusura e commit

- Gli errori del driver sono eccezioni di tipo *mysql.connector.Error*: si gestiscono con *try / except*
- Con l'istruzione *with* connessione e cursor vengono chiusi automaticamente al termine del blocco
- **Attenzione:** a differenza di JDBC, l'autocommit è disattivato di default
  - Le modifiche (*INSERT*, *UPDATE*, *DELETE*) vanno confermate con *conn.commit()*, altrimenti vengono perse alla chiusura della connessione

Creare una tabella

```
import mysql.connector

with mysql.connector.connect(
        host="%INDIRIZZO%", database="%DATABASET%",
        user="%UTENTE%", password="%PASSWORD%") as conn:
    with conn.cursor() as cur:
        sql = """CREATE TABLE student (
            id INTEGER NOT NULL,
            name VARCHAR(255),
            surname VARCHAR(255),
            birthplace VARCHAR(255),
            PRIMARY KEY (id))"""
        cur.execute(sql)
        print("Tabella creata!")
```

Inserire valori con execute()

```
query = ("INSERT INTO student (id, name, surname, birthplace) "
        "VALUES (154247, 'Mario', 'Neri', 'Bolzano')")
with conn.cursor() as cur:
        cur.execute(query)
conn.commit()
print("Inserimento effettuato!")
```

Inserire valori con parametri

```
query = ("INSERT INTO student (id, name, surname, birthplace) "
        "VALUES (%s, %s, %s, %s)")
with conn.cursor() as cur:
        cur.execute(query, (128853, "Francesco", "Neri", "Rovereto"))
conn.commit()
```

- *%s* è il segnaposto per qualsiasi tipo di dato (non solo per le stringhe), come il *?* di PreparedStatement
- Il driver formatta i valori: mai costruire le query concatenando stringhe o con f-string (SQL injection)

Cursor e risultati

- Le query si eseguono con un *cursor*, creato dalla connessione: *conn.cursor()*
- Dopo *execute()* di una SELECT, il cursor si può scorrere con un ciclo *for*: ogni riga è una tupla, ad es. *(154247, "Mario", "Neri", "Bolzano")*
- *fetchone()* restituisce una riga, *fetchall()* tutte le righe in una lista
- Con *conn.cursor(dictionary=True)* ogni riga è un dizionario: *row["name"]*, come *getString("name")* in JDBC

Python Query

```
with conn.cursor() as cur:
        cur.execute("SELECT * FROM student")
        for (student_id, name, surname, birthplace) in cur:
                print(f"ID: {student_id} - Nome: {name}"
                        f" - Cognome: {surname}"
                        f" - Luogo di nascita: {birthplace}")
```

Python Query con parametri

```
surname = "Neri"
with conn.cursor(dictionary=True) as cur:
    cur.execute("SELECT * FROM student WHERE surname = %s",
        (surname,))
    for row in cur:
        print(f"ID: {row['id']} - Nome: {row['name']}"
            f" - Cognome: {row['surname']}"
            f" - Luogo di nascita: {row['birthplace']}")
```

- *(surname,)* è una tupla con un solo elemento: la virgola è obbligatoria

Aggiornare il database

```
surname = "Bianchi"
with conn.cursor() as cur:
        cur.execute("DELETE FROM student WHERE surname = %s",
                (surname,))
        print(cur.rowcount, "righe eliminate")
conn.commit()
```

JDBC e Python a confronto

	JDBC (Java)	Python
Driver	*Connector/J*	*Connector/Python*
Connessione	*DriverManager.getConnection(url, props)*	*mysql.connector.connect(host=..., user=..., ...)*
Esecuzione	*Statement / PreparedStatement*	*cursor.execute(query, parametri)*
Segnaposto	*?*	*%s*
Risultati	*ResultSet: next(), getString("name")*	*ciclo for sul cursor: row["name"]*
Modifiche	*autocommit attivo*	*autocommit disattivo: conn.commit()*
Chiusura	*close()*	*with (chiusura automatica)*

Introduzione all'uso di MySQL/MariaDB e di DBeaver

Basi di dati

Riccardo Billero
riccardo.billero@unitn.it
A.A. 2026/2027

UNIVERSITÀ DI TRENTO